Polymorphic Bottom-up Weighted Relational Programming

Dmitri Volkov

May 2, 2026

Abstract

This work demonstrates a new approach for implementing polymorphism for bottom-up relational languages, without monomorphization. We begin presenting semiringKanren, a bottom-up weighted relational programming language. We extend this base language to support polymorphism. We describe a new method to compile polymorphic semiringKanren programs into non-polymorphic ones, based on *equality patterns* and *large-enough instances* of polymorphic relations. We prove the correctness of this method. Finally, we consider existing work and suggest directions for future research.

Contents

1 Introduction **3**

2 semiringKanren **5**
2.1 Example Programs . . . 5
2.1.1 Relations . . . 5
2.1.2 Weighted Relations . . . 6
2.1.3 Recursion . . . 6
2.1.4 Semirings . . . 7
2.2 Syntax and Typing . . . 8
2.3 Semantics . . . 11

3 Polymorphic semiringKanren **14**
3.1 Polymorphic Relations . . . 14
3.1.1 Equal . . . 14
3.1.2 Sum-Swap . . . 15
3.1.3 Option-Map . . . 16
3.1.4 Two-Valued . . . 16
3.2 Syntax and Typing . . . 17
3.3 Monomorphizing Semantics . . . 20
3.4 Non-Monomorphizing Semantics . . . 22
3.4.1 Equality Patterns . . . 22
3.4.2 Large-Enough Relation Instances . . . 24
3.4.3 Compiling Polymorphic Programs . . . 25
3.5 Non-Monomorphizing Semantics Correctness . . . 26
3.5.1 Equality Patterns Preserve Weight . . . 26
3.5.2 Compiling Polymorphic Relation Calls . . . 34

4 Related Work **39**
4.1 miniKanren . . . 39
4.2 Datalog . . . 40
4.3 Weighted Logic Languages . . . 40
4.4 Pixel Arrays . . . 41
4.5 SAT Compilation . . . 41
4.6 Polymorphic Logic Languages . . . 41

4.7 Other Related Work 42

5 Conclusions and Future Work **43**
5.1 Future Work 44
5.1.1 Large-Enough Relation Instances 44
5.1.2 Higher-Order Relations 44
5.1.3 Recursive Types 44
5.1.4 Semiring Fixpoint Convergence 45
5.1.5 SMT Solving 45
5.1.6 Applications 46
5.1.7 Alternative Evaluation Strategies 46

Bibliography **47**

Chapter 1

Introduction

Relational programming (also known as logic programming) is arguably the most expressive and powerful paradigm for constructing computations. Where most languages express programs as sequences of instructions or mathematical functions, relational languages express programs with the full power of mathematical relations. Applications of relational programming languages include natural language processing[21] and program generation[5], both of which can be difficult to express in other paradigms.

Relational programming languages are usually implemented either via *top-down* search, or *bottom-up* fact collection. Top-down relational programming languages include Prolog and miniKanren; bottom-up relational languages include datalog and SQL. The top-down approach can effectively handle recursive data, but can also be prone to getting stuck searching an ineffectual branch. The bottom-up approach may not be able to handle recursive data, and may use space less efficiently, but has a much clearer semantics (especially for weighted relations) and can avoid many of the traps of top-down languages.

For programming languages, *polymorphism* is used to describe code that can operate on different types of data. When programmers want to processes different types of data in similar ways, polymorphism allows them to do so without manually re-writing similar code for each particular type. Polymorphism is generally classified as *parametric* or *ad-hoc*. In parametric polymorphism, unknwon types are expressed as *type variables*, and direct operations on values of unknown types are prohibited. Ad-hoc polymorphism does allow operations, and these operations behave differently depending on the underlying types of the values. In this work, we focus on parametric polymorphism.

The notion of a bottom-up polymorphic relational programming language can feel paradoxical. How can we collect facts like "x is related to y" when we do not know the domains x and y are drawn from? One approach might be to track which types for x and y are actually used in practice, and create versions of the relations specialized for those types. This approach is known as *monomorphization*. However, this can substantially increase the size of programs, and does not generalize. What if we want to call a relation on new types,

for which specialized relations have not yet been generated?

In this work, we present semiringKanren, a bottom-up weighted relational programming language supporting polymorphism *without* monomorphization. We begin by showcasing the base semiringKanren langauge, which does not support polymorphism, to build familiarity and establish the foundational concepts. We then demonstrate how the syntax and type system of this base language can be extended to support polymorphism. We briefly describe a monomorphizing semantics for this language, before introducing a non-monomorphizing compilation technique from the polymorphic language to the base language. In particular, we show how *equality patterns* can be used to extract the underlying structure of *large-enough instances* of relations to handle polymorphic relation calls at different types. We conclude the section with correctness proofs of this approach. We compare semiringKanren with existing work, with a particular focus on other polymorphic bottom-up weighted relational languages. Finally, we consider areas for further exploration and development.

We believe that this new approach to polymorphism can offer improvements in the broader space of relational programming. In particular, we hope it may offer efficiency gains, and enable "separate evalaution" of polymorphic programs.

Chapter 2

semiringKanren

Here, we provide an introduction to the semiringKanren language and the underlying principles. The semiringKanren language consists of relations defined in terms of goals, which may take in values corresponding to algebraic data types. Relations take arguments, and semiringKanren computes weights for every possible value assignment to arguments.

The semiringKanren language as presented here differs from prior work [29]. Rather than having primitive "constructor" relations, this version extends the value language allowed within unification and disunification. We believe this makes programs easier to write, and simplifies the implementation of algorithms occuring later in this work.

2.1 Example Programs

We introduce the semiringKanren language through examples.

2.1.1 Relations

In semiringKanren, programs are made up of relations, which in turn are made up of goals. Relations take zero or more typed argument variables, and either *succeed* or *fail* for each possible assignment of values to those variables. We use `defrel` to define new relations.

```
(defrel (coin-flip (coin : (Sum Unit Unit)))
  (disj
    (== coin (left sole))
    (== coin (right sole))))
```
(2.1)

Here, we define a relation `coin-flip` which takes a single argument of type `(Sum Unit Unit)`. The type `(Sum Unit Unit)` has two values: `(left sole)` and `(right sole)`. We use `(left sole)` to represent tails, and `(right sole)`

to represent heads. This definition showcases two types of goals: the connective `disj` and the primitive relation `==`. The `disj` goal succeeds when either of its subgoals succeeds. The primitive relation `==` succeeds when its two arguments are equal. Thus, this relation succeeds either when it is called with the argument `(left sole)`, or with the argument `(right sole)`.

Note that many relational programming languages have a distinction between relations and *queries* on those relations. This distinction is not neccesary in semiringKanren because relation evaluation already directly returns the satisfying argument values.

2.1.2 Weighted Relations

Beyond success and failure, semiringKanren relations can have different *weights*. We can add weight to a program branch with the `factor` goal. We can modify the previous example to express a weighted unfair coin flip:

```
(defrel (unfair-coin-flip (coin : (Sum Unit Unit)))
  (disj
    (conj (factor 0.7) (== coin (left sole)))
    (conj (factor 0.3) (== coin (right sole)))))
```
(2.2)

We can think of `conj` as succeeding when both of its subgoals succeed. Success corresponds to a weight of 1, and failure corresponds to a weight of 0. When working with weights, `conj` generalizes to multiplication and `disj` generalizes to addition. Hence, this relation assigns the weight 0.7 to `(left sole)` (corresponding to heads), and the weight 0.3 to `(right sole)` (corresponding to tails).

2.1.3 Recursion

Within a relation, we can call other relations or even recursively call the current relation. For example, we can simulate a fair coin flip with unfair coins [30]. For this process, we flip an unfair coin twice. If the two coin flips have the same result, we repeat the process. Otherwise, we go with the result of the first coin

flip.

```
(defrel (fair-coin-clip (coin : (Sum Unit Unit)))
  (fresh ((coin1 : (Sum Unit Unit))
          (coin2 : (Sum Unit Unit)))
    (conj
      (unfair-coin-flip coin1)
      (unfair-coin-flip coin2)
      (disj
        (conj
          (== coin1 coin2)
          (fair-coin-flip coin)))
        (conj
          (=/= coin1 coin2)
          (== coin coin1)))))
```
(2.3)

Here, we use the `fresh` goal, which introduces *fresh variables* and sums over the subgoal weights for each value the variables can take. We also use the disequality primitive relation `=/=`, which tests that its arguments are not equal.

The semiringKanren languages handles recursion by finding the *least fixed point* of relations. This means it repeatedly evaluates each relation, using the weights from the previous evaluation each time there is a call. Throughout this process, relations gradually accumulate more information. We refer to this as *bottom-up evaluation*.

2.1.4 Semirings

So far we have been working with the real numbers $\mathbb{R}$, where `disj` corresponds to real number addition, and `conj` corresponds to real number multiplication. However, semiringKanren can evaluate programs over any *commutative semiring*. A commutative semiring is a set equipped with $+$ and $\times$ operations, both of which are commutative and associative and obey distributivity $a \times (b + c) = (a \times b) + (a \times c)$. We also require this set to have a multiplicative identity 1, and an additive identity and multiplicative annihilator 0. Sometimes the term *rig*, as in "ri<u>n</u>g without <u>n</u>egation," is used. Besides the real numbers equipped with addition and multiplication $(\mathbb{R}, +, *, 0, 1)$, other semirings include the *boolean semiring*, equipped with logical "or" and logical "and" $(\mathbb{B}, \vee, \wedge, \bot, \top)$, and the *min-tropical semiring*: the real numbers and infinity equipped with minimum and addition $(\mathbb{R}^\infty, \min, +, \infty, 0)$.

To show the utility of different semirings, we present the example of transitive closure of a relation. Transitive closure can be thought of as "reachability by chaining." Given a relation $a\ R\ b$, for each element x, we want to find all y such that $x\ R \ldots R\ y$. We can represent the relation we want to process as a graph:

Here, our relation operates on numbers 0 through 3. For convenience, we call this type 4, and represent the members of this type with $\mathtt{0}_4, \mathtt{1}_4, \mathtt{1}_4, \mathtt{3}_4$. In practice, we create this type using sum of four `Units`. Assuming we give each edge in the graph some weight w, we can write this relation in semiringKanren as follows:

```
(defrel (graph ((x : 4) (y : 4)))
  (disj
    (conj (== x 0₄) (== y 1₄) (factor w))
    (conj (== x 1₄) (== y 0₄) (factor w))
    (conj (== x 1₄) (== y 2₄) (factor w))
    (conj (== x 3₄) (== y 2₄) (factor w))))
```
(2.4)

We can now express reachability/transitive closure as a recursive relation:

```
(defrel (connect ((x : 4) (y : 4)))
  (disj
    (graph x y)
    (fresh ((z : 4))
      (conj
        (connect x z)
        (connect z y)))))
```
(2.5)

When we evaluate this over the boolean semiring with $w = \top$, we get reachability:

	$y = \mathtt{0}_4$	$y = \mathtt{1}_4$	$y = \mathtt{2}_4$	$y = \mathtt{3}_4$
$x = \mathtt{0}_4$	$\top$	$\top$	$\top$	$\bot$
$x = \mathtt{1}_4$	$\top$	$\top$	$\top$	$\bot$
$x = \mathtt{2}_4$	$\bot$	$\bot$	$\bot$	$\bot$
$x = \mathtt{3}_4$	$\bot$	$\bot$	$\top$	$\bot$

When we evaluate this over the min-tropical semiring with $w = 1$, we get the shortest path lengths:

	$y = \mathtt{0}_4$	$y = \mathtt{1}_4$	$y = \mathtt{2}_4$	$y = \mathtt{3}_4$
$x = \mathtt{0}_4$	2	1	2	∞
$x = \mathtt{1}_4$	1	2	1	∞
$x = \mathtt{2}_4$	∞	∞	∞	∞
$x = \mathtt{3}_4$	∞	∞	1	∞

2.2 Syntax and Typing

We give the syntax of semiringKanren, ommitting some syntactic sugar, in figure 2.1. Here, `conj` and `disj` must have two subgoals, and `fresh` can only introduce a single variable at a time.

Programs	P	$::=$	$Rel \ldots$
Relations	Rel	$::=$	$\texttt{(defrel (}R\ \texttt{(}x : \tau\texttt{)}\ \ldots\texttt{)}\ g\texttt{)}$
Goals	g	$::=$	$\texttt{(conj}\ g\ g\texttt{)}$
		$\mid$	$\texttt{(disj}\ g\ g\texttt{)}$
		$\mid$	$\texttt{(fresh (}\texttt{(}x : \tau\texttt{)}\ \ldots\texttt{)}\ g\texttt{)}$
		$\mid$	$\texttt{(==}\ v\ v\texttt{)}$
		$\mid$	$\texttt{(=/=}\ v\ v\texttt{)}$
		$\mid$	$\texttt{(}R\ v\ \ldots\texttt{)}$
		$\mid$	$\texttt{(factor}\ k\texttt{)}$
Values	v	$::=$	$\texttt{sole}$
		$\mid$	$\texttt{(left}_{\tau_1,\tau_2}\ v\texttt{)}$
		$\mid$	$\texttt{(right}_{\tau_1,\tau_2}\ v\texttt{)}$
		$\mid$	$\texttt{(pair}\ v\ v\texttt{)}$
		$\mid$	x
Types	τ	$::=$	$\texttt{Unit}$
		$\mid$	$\texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}$
		$\mid$	$\texttt{(Pair}\ \tau_1\ \tau_2\texttt{)}$
Relation names	R		
Variables	x, y, z		
Weights	r	$\in$	$\mathbb{K}$

Figure 2.1: Syntax for semiringKanren.

$$\frac{}{\Delta \vdash \texttt{sole} : \texttt{Unit}} \qquad \frac{\Delta \vdash v : \tau_1}{\Delta \vdash \texttt{(left}_{\tau_1+\tau_2}\ v\texttt{)} : \texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}}$$

$$\frac{\Delta \vdash v : \tau_2}{\Delta \vdash \texttt{(right}_{\tau_1+\tau_2}\ v\texttt{)} : \texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}} \qquad \frac{\Delta \vdash v_1 : \tau_1 \qquad \Delta \vdash v_2 : \tau_2}{\Delta \vdash \texttt{(pair}\ v_1\ v_2\texttt{)} : \texttt{(Prod}\ \tau_1\ \tau_2\texttt{)}}$$

$$\frac{}{\Delta, x : \tau \vdash x : \tau}$$

Figure 2.2: Inference rules for value typing judgement.

$$\frac{\Gamma;\Delta \vdash g_1 \text{ wt} \qquad \Gamma;\Delta \vdash g_2 \text{ wt}}{\Gamma;\Delta \vdash \texttt{(conj } g_1 \ g_2\texttt{)} \text{ wt}} \qquad \frac{\Gamma;\Delta \vdash g_1 \text{ wt} \qquad \Gamma;\Delta \vdash g_2 \text{ wt}}{\Gamma;\Delta \vdash \texttt{(disj } g_1 \ g_2\texttt{)} \text{ wt}}$$

$$\frac{\tau \text{ vt} \qquad \Gamma;\Delta, x:\tau \vdash g \text{ wt}}{\Gamma;\Delta \vdash \texttt{(fresh ((}x:\tau\texttt{)) } g\texttt{)} \text{ wt}} \qquad \frac{\Delta \vdash v_1 : \tau \qquad \Delta \vdash v_2 : \tau}{\Gamma;\Delta \vdash \texttt{(==} \ v_1 \ v_2\texttt{)} \text{ wt}}$$

$$\frac{\Delta \vdash v_1 : \tau \qquad \Delta \vdash v_2 : \tau}{\Gamma;\Delta \vdash \texttt{(=/=} \ v_1 \ v_2\texttt{)} \text{ wt}} \qquad \frac{\Delta \vdash \vec{v} : \vec{\tau}}{\Gamma, (R : \vec{\tau} \rightarrow); \Delta \vdash \texttt{(}R \ \vec{v}\texttt{)} \text{ wt}}$$

$$\frac{}{\Gamma;\Delta \vdash \texttt{(factor } r\texttt{)} \text{ wt}}$$

Figure 2.3: Inference rules for goal typing judgement.

$$\frac{\Gamma;\vec{x}:\vec{\tau} \vdash g \text{ wt}}{\Gamma \vdash \texttt{(defrel (}R \ \texttt{(}\vec{x}:\vec{\tau}\texttt{))} \ g\texttt{))} \text{ wt}}$$

$$\frac{\begin{array}{c}(R_1 : \vec{\tau}_1 \rightarrow), \ldots, (R_N : \vec{\tau}_N \rightarrow) \vdash \texttt{(defrel (}R_1 \ \texttt{(}\vec{x}_1 : \vec{\tau}_1\texttt{))} \ g_1\texttt{)} \text{ wt} \\ \vdots \\ (R_1 : \vec{\tau}_1 \rightarrow), \ldots, (R_N : \vec{\tau}_N \rightarrow) \vdash \texttt{(defrel (}R_N \ \texttt{(}\vec{x}_N : \vec{\tau}_N\texttt{))} \ g_1\texttt{)} \text{ wt}\end{array}}{\begin{array}{c}\texttt{(defrel (}R_1 \ \texttt{(}\vec{x}_1 : \vec{\tau}_1\texttt{))} \ g_1\texttt{)), }\ldots\texttt{,} \\ \texttt{(defrel (}R_N \ \texttt{(}\vec{x}_N : \vec{\tau}_N\texttt{))} \ g_N\texttt{))} \text{ wt}\end{array}}$$

Figure 2.4: Inference rules for relation and program typing judgements.

$$
\begin{aligned}
[\![\texttt{Unit}]\!] &= \{\texttt{sole}\} \\
[\![\texttt{(Sum}\ \tau_1\ \ \tau_2\texttt{)}]\!] &= \{\texttt{(left}\ \ v\texttt{)} \mid v \in [\![\tau_1]\!]\} \cup \{\texttt{(right}\ \ v\texttt{)} \mid v \in [\![\tau_2]\!]\} \\
[\![\texttt{(Prod}\ \tau_1\ \ \tau_2\texttt{)}]\!] &= \{\texttt{(pair}\ \ v_1\ \ v_2\texttt{)} \mid v_1 \in [\![\tau_1]\!], v_2 \in [\![\tau_2]\!]\}
\end{aligned}
$$

Figure 2.5: Denotational semantics for types.

The type system for semiringKanren is shown in figure 2.2 which shows the type system for values, figure 2.3 which shows the type system for goals, and figure 2.4 which shows the type system for relations and programs. For values, the typing judgement $\Delta \vdash v : \tau$ means that the value v has type τ under the value context Δ, which tracks the types of variables. Note that we annotated the `left` and `right` sum type constructors with full sum types, to disambiguate the larger type while type checking. In practice, this is not necessary. Each entry in Δ has the form $x : \tau$, where x is a variable (from either a relation argument, or `fresh`), and τ is a type. For goals, we use the judgement $\Gamma; \Delta \vdash g$ wt to indicate that the goal g is well-typed under value context Δ and relation context Γ, which tracks the types of relations. Each entry in Γ has the form $(R : \tau_1, \ldots, \tau_n \rightarrow)$ which indicates that the relation with name R takes n arguments, which have types $\tau_1, \ldots, \tau_n$. For relations, we use the similar judgement $\Gamma \vdash \mathit{Rel}$ wt, ommitting the Δ context because it is not yet needed (there are no variables to bind). For programs, where there is no larger context, we simply use P wt.

Note that our relation typing judgement is "the relation is well-typed under a context" rather than "the relation has type ...". Relations are required to specify all needed typing information, so it only is necessary to check that their bodies are compatible. Because the types of all relations are known ahead-of-time, we avoid potential issues of recursive dependencies when type-checking relation calls. We use the shorthand $(\vec{x} : \vec{\tau})$ as shorthand for $(x_1 : \tau_1)\ \ldots\ (x_n : \tau_n)$, or similar.

2.3 Semantics

Each type τ denotes a finite set of values, as shown in figure 2.5. We can use values as indices into an array. One way this can be done is by ordering the values lexicographically, then using their position in the list as an index drawn from $\mathbb{N}$.

Definition 1. We define the *size of type* τ, notated $|\tau|$, to be the number of values of type τ.

$$|\tau| = |[\![\tau]\!]|$$

$$
\begin{aligned}
[\![\texttt{sole}]\!](\delta) &= \texttt{sole} \\
[\![\texttt{(left } v\texttt{)}]\!](\delta) &= \texttt{(left } [\![v]\!](\delta)\texttt{)} \\
[\![\texttt{(right } v\texttt{)}]\!](\delta) &= \texttt{(right } [\![v]\!](\delta)\texttt{)} \\
[\![\texttt{(pair } v_1 \ v_2\texttt{)}]\!](\delta) &= \texttt{(pair } [\![v_1]\!](\delta) \ [\![v_2]\!](\delta)\texttt{)} \\
[\![x]\!](\delta) &= \delta(x)
\end{aligned}
$$

Figure 2.6: Denotational semantics for values.

$$
\begin{aligned}
[\![\texttt{(conj } g_1 \ g_2\texttt{)}]\!](\gamma;\delta) &= [\![g_1]\!](\gamma;\delta) \times [\![g_2]\!](\gamma;\delta) \\
[\![\texttt{(disj } g_1 \ g_2\texttt{)}]\!](\gamma;\delta) &= [\![g_1]\!](\gamma;\delta) + [\![g_2]\!](\gamma;\delta) \\
[\![\texttt{(fresh ((}x : \tau\texttt{)) } g\texttt{)}]\!](\gamma;\delta) &= \sum_{v \in [\![\tau]\!]} [\![g]\!](\gamma;\delta, x \mapsto v) \\
[\![\texttt{(==} \ v_1 \ v_2\texttt{)}]\!](\gamma;\delta) &= \begin{cases} 1 & \text{if } [\![v_1]\!](\delta) = [\![v_2]\!](\delta) \\ 0 & \text{if } [\![v_1]\!](\delta) \neq [\![v_2]\!](\delta) \end{cases} \\
[\![\texttt{(=/=} \ v_1 \ v_2\texttt{)}]\!](\gamma;\delta) &= \begin{cases} 1 & \text{if } [\![v_1]\!](\delta) \neq [\![v_2]\!](\delta) \\ 0 & \text{if } [\![v_1]\!](\delta) = [\![v_2]\!](\delta) \end{cases} \\
[\![\texttt{(}R \ \vec{v}\texttt{)}]\!](\gamma;\delta) &= \gamma(R)([\![\vec{v}]\!](\delta)) \text{ where } \vec{v} : \vec{\tau} \\
[\![\texttt{(factor } r\texttt{)}]\!](\gamma;\delta) &= r
\end{aligned}
$$

Figure 2.7: Denotational semantics for goals.

The semantics for values, as shown in figure 2.6, is similar to identity but substitutes variables where appropriate. We use the term *concrete values* for values which do not include variables. The denotation for values turns arbitrary values (which may contain variables) into concrete values. To this end, we take a *value environment* δ, consisting of entries of the form $x \mapsto v$, where x is a variable name and v is a concrete value. For δ to be valid, each entry $x \mapsto v$ should have a corresponding type entry in Δ where $\Delta \vdash x : \tau$ and $\cdot \vdash v : \tau$. In the denotational semantics for values, when we reach a variable, we apply δ to get its concrete value.

The core semiringKanren semantics is that of goals. For the denotation of goals, we take a *relation environment* γ mapping relation names to precomputed relations (whose types correspond to Γ), as well as a value environment δ. The relational environment γ is valid if for any $(R : \tau_1, \ldots, \tau_n \ \rightarrow) \in \Gamma$, then the relation in γ corresponding to relation name R maps indices drawn from $[\![\tau_1]\!] \times \cdots \times [\![\tau_n]\!]$ to weights drawn from $\mathbb{K}$.

The denotation of goals as shown in figure 2.7 computes a weight depending on the relation environment and current value environment. Goals can be

$$[\![\texttt{(defrel (}R\ \texttt{(}\vec{x}:\vec{\tau}\texttt{)}\ \texttt{g)}]\!](\gamma) = (\vec{i}) \mapsto [\![g]\!](\gamma; \vec{x} \mapsto [\![\vec{i}]\!]; \sigma)$$
$$[\![Rel_1, \ldots, Rel_Q]\!] = \text{fix } \gamma \text{ in } R_1 \mapsto [\![Rel_1]\!](\gamma), \ldots, R_Q \mapsto [\![Rel_Q]\!](\gamma)$$

Figure 2.8: Denotational semantics for relations and programs.

understood as operating on arrays indexed by δ and parameterized by γ. For example, `conj` and `disj` correspond to element-wise multiplication and addition, and `(factor` r`)` corresponds to producing an array with the constant r in every entry. When x and y are different variables, `(==` x y`)` produces an array with ones along the diagonal corresponding to the x and y dimensions, and `(==` x y`)` relatedly produces an array with ones on the off-diagonal and zeros along the diagonal. The `fresh` goal sums along one dimension of a rank-$(n+1)$ array (corresponding to the dimension of the new variable) to produce a rank-n array.

We define the denotation of a relation to be a map from elements drawn from the types of its arguments, to weights. Thus, the elements of γ are pre-computed relation denotations. We compute this denotation by evaluating the goal structure of the relation for all possible values inhabiting the argument types We now define the denotation of a complete program to be the least fixed point of the denotation of all relations. So we use γ to repeatedly compute the denotation of each relation to get a new γ, and repeat this process until γ stabilizes. In practice, γ starts out consisting of relations which always fail. Then, the fixpointing process can be thought of as iteratively deriving new information about γ. These semantics are shown in figure 2.8

Chapter 3

Polymorphic semiringKanren

3.1 Polymorphic Relations

We extend semiringKanren with *polymorphic relations*, which operate on values of unknown types (represented with *type variables*). We introduce polymorphic relations by example.

3.1.1 Equal

Consider the following relation:

```
(defrel (equal (x : α) (y : α))
  (== x y))
```

This relation takes in two values of the same variable type, and checks that they are equal. We can call this relation at different types. For example, following call succeeds:

```
(equal sole sole)
```

while the following call fails:

```
(equal (pair (left sole) (right sole)) (pair (right sole) (left sole)))
```

In the first example, α gets mapped to the type `Unit` In the second example, α gets mapped to the type `(Prod (Sum Unit Unit) (Sum Unit Unit))`. Note that the primitive relations `==` and `=/=` are polymorphic, even in the base language; they can operate on any type, including variable types, as long as both arguments have the same type.

3.1.2 Sum-Swap

Continuing to a larger example:

```
(defrel (sum-swap (x : (Sum α β)) (y : (Sum β α)))
  (disj
    (fresh ((a : α))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh ((b : β))
      (conj
        (== x (right b))
        (== y (left b))))))
```

This relation replaces top-level `lefts` in a sum type with `rights`, and vice versa. We can have multiple type variables, and include type variables within larger type structures. Assuming we call `sum-swap` with α = `Unit` and β = `(Sum Unit Unit)`,

```
(sum-swap (left sole) q)
```

succeeds when q = `(right sole)`, and

```
(sum-swap (right (right sole)) q)
```

succeeds when q = `(left (right sole))`. When we have multiple type variables, they can be mapped to different types.

3.1.3 Option-Map

As a "real-world" motivating example, consider the following:

```
(defrel (option-map
  (f : (Prod α β)) (x : (Sum Unit α)) (y : (Sum Unit β)))
  (disj
    (conj
      (== x (left sole))
      (== y (left sole)))
    (fresh ((a : α) (b : β))
      (conj
        (== x (right a))
        (== f (pair a b))
        (== y (right b))))))
```

The type `(Sum Unit` α`)` can be thought of as an "option type" where `(right` v`)` can be thought of carrying a value, and `(left sole)` can be though of not carrying a value, or "null." Functional analogs of `option-map` are common in typed functional programming languages when working with optional values. The relation `option-map` takes a relation call encoded as a pair, and applies it to the "value" part of x and y if possible. For example, `option-map` may be called as follows:

```
(fresh ((h : (Sum Unit Unit)) (k : Sum Unit Unit))
  (conj
    (sum-swap h k)
    (option-map (pair h k) (right (left sole)) q)))
```

Which would succeed for $q =$ `(right (right sole))`. Here, both α and β are mapped to `(Sum Unit Unit)` within the `option-map` call.

3.1.4 Two-Valued

We present one more example:

```
(two-valued (x : α)
  (fresh ((y : α))
    (=/= x y)))
```

This relation succeeds when it can find a value of the argument's type disequal from the argument. For example, assuming α gets mapped to `Sum Unit Unit` the following succeeds:

```
two-valued (left sole))
```

because it can find y with the value `(right sole)`. However, consider calling `two-valued` with α mapped to `Unit`:

```
(two-valued sole)
```

This call fails, because the only value of type `Unit` is `sole`, and `(=/= sole sole)` fails. Success/failure depends on the type of the argument of `two-value` rather than the value.

The obvious way to implement polymorphism is to generate multiple instances of each relation for each type variable mapping that gets used. This is known as *monomorphization*, and is historically the only way polymorphism has been implemented for bottom-up relational languages. With semiringKanren, we present a new method of implementing polymorphism based on *large-enough instances* of polymorphic relations, where polymorphic relation calls with arbitrary type mappings are compiled to only use a single type mapping, with extra code generated at call sites to reconstruct the desired types.

We first present the syntax and typing rules for semiringKanren extended with polymorphic relations. Next, we provide a semantics for polymorphic semiringKanren based on monomorphization. Finally, we show how polymorphic relation calls can be compiled to use large-enough instances, and prove the correctness of this approach.

3.2 Syntax and Typing

We expand the syntax and typing of minimal semiringKanren to support polymorphic relations. We give the syntax of polymorphic semiringKanren in figure 3.1.

Here, the Γ context holds relation types, and the Δ context holds variable types. As before, we use $\Delta \vdash v : \tau$ to denote the value (and therefore variable) typing judgement asserting that value v has type τ. The value typing judgement is shown in figure 3.3. The $\Gamma; \Delta \vdash g$ wt judgement denotes that the goal g is well-typed under the Γ and Δ contexts, as shown in figure 3.4. The $\Gamma \vdash R$ wt judgement denotes that the relation R is well-typed under the Γ context, and the P wt judgement denotes that the entire program is well-typed. The relation and program typing judgements are shown in figure 3.5. We introduce a new $\Delta \vdash \tau$ vt judgement (defined in figure **??**), which asserts that the type τ is valid under the Δ context. The judgements within the Γ context are extended to support type variables: $(R : \forall \alpha_1, \ldots, \alpha_m . \tau_1, \ldots, \tau_n \rightarrow)$. As before, Δ holds value typing judgements (of the form $x : \tau$). Δ is also extended to hold judgements of the form $\alpha : *$, indicating that the type variable α can be treated as a valid type in the current context. In both cases, type variables may now appear in τ.

In the formal semantics type variables are explicitly specified, but in practice they can be automatically extracted by looking at the argument types. Also note the use of a type variable substitution, σ, when checking the validity of types which may contain not-yet-valid type variables (such as within polymorphic

Programs	P	$::=$	$Rel\ldots$
Relations	Rel	$::=$	`(defrel (`R $\forall\alpha\ldots$ `.` $(x:\tau)$ $\ldots$`)` g`)`
Goals	g	$::=$	`(conj` g g`)`
		$\mid$	`(disj` g g`)`
		$\mid$	`(fresh (`$(x:\tau)$ $\ldots$`)` g`)`
		$\mid$	`(==` v v`)`
		$\mid$	`(=/=` v v`)`
		$\mid$	`(`R v $\ldots$`)`
		$\mid$	`(factor` k`)`
Values	v	$::=$	`sole`
		$\mid$	`(left`$_{\tau_1,\tau_2}$ v`)`
		$\mid$	`(right`$_{\tau_1,\tau_2}$ v`)`
		$\mid$	`(pair` v v`)`
		$\mid$	x
Types	τ	$::=$	`Unit`
		$\mid$	`(Sum` τ_1 τ_2`)`
		$\mid$	`(Pair` τ_1 τ_2`)`
		$\mid$	α
Relation names	R		
Variables	x, y, z		
Type variables	α, β		
Weights	r	$\in$	$\mathbb{K}$

Figure 3.1: Syntax for polymorphic semiringKanren

$$\frac{}{\Delta \vdash \texttt{Unit}\ \text{vt}} \qquad \frac{\Delta \vdash \tau_1\ \text{vt} \qquad \Delta \vdash \tau_2\ \text{vt}}{\Delta \vdash \texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}\ \text{vt}} \qquad \frac{\Delta \vdash \tau_1\ \text{vt} \qquad \Delta \vdash \tau_2\ \text{vt}}{\Delta \vdash \texttt{(Prod}\ \tau_1\ \tau_2\texttt{)}\ \text{vt}}$$

$$\frac{}{\Delta, \alpha : * \vdash \alpha\ \text{vt}}$$

Figure 3.2: Inference rules for polymorphic type validity judgement.

$$\frac{}{\Delta \vdash \texttt{sole} : \texttt{Unit}} \qquad \frac{\Delta \vdash v : \tau_1}{\Delta \vdash \texttt{(left}_{\tau_1+\tau_2}\ v\texttt{)} : \texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}}$$

$$\frac{\Delta \vdash v : \tau_2}{\Delta \vdash \texttt{(right}_{\tau_1+\tau_2}\ v\texttt{)} : \texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}} \qquad \frac{\Delta \vdash v_1 : \tau_1 \qquad \Delta \vdash v_2 : \tau_2}{\Delta \vdash \texttt{(pair}\ v_1\ v_2\texttt{)} : \texttt{(Prod}\ \tau_1\ \tau_2\texttt{)}}$$

$$\frac{\Delta, x : \tau \vdash \tau \text{ vt}}{\Delta, x : \tau \vdash x : \tau}$$

Figure 3.3: Inference rules for polymorphic value typing judgement.

$$\frac{\Gamma; \Delta \vdash g_1 \text{ wt} \qquad \Gamma; \Delta \vdash g_2 \text{ wt}}{\Gamma; \Delta \vdash \texttt{(conj}\ g_1\ g_2\texttt{)} \text{ wt}} \qquad \frac{\Gamma; \Delta \vdash g_1 \text{ wt} \qquad \Gamma; \Delta \vdash g_2 \text{ wt}}{\Gamma; \Delta \vdash \texttt{(disj}\ g_1\ g_2\texttt{)} \text{ wt}}$$

$$\frac{\Delta \vdash \tau \text{ vt} \qquad \Gamma; \Delta, x : \tau \vdash g \text{ wt}}{\Gamma; \Delta \vdash \texttt{(fresh (}(x : \tau)\texttt{)}\ g\texttt{)} \text{ wt}} \qquad \frac{\Delta \vdash v_1 : \tau \qquad \Delta \vdash v_2 : \tau}{\Gamma; \Delta \vdash \texttt{(==}\ v_1\ v_2\texttt{)} \text{ wt}}$$

$$\frac{\Delta \vdash v_1 : \tau \qquad \Delta \vdash v_2 : \tau}{\Gamma; \Delta \vdash \texttt{(=/=}\ v_1\ v_2\texttt{)} \text{ wt}} \qquad \frac{\Delta \vdash \vec{v} : \vec{\tau}}{\Gamma, (R : \vec{\tau} \rightarrow); \Delta \vdash \texttt{(}R\ \vec{v}\texttt{)} \text{ wt}}$$

$$\frac{}{\Gamma; \Delta \vdash \texttt{(factor}\ r\texttt{)} \text{ wt}}$$

Figure 3.4: Inference rules for polymorphic goal typing judgement.

$$\frac{\Gamma; \vec{\alpha} : *, \vec{x} : \vec{\tau} \vdash g \text{ wt}}{\Gamma \vdash \texttt{(defrel (}R\ \forall\vec{\alpha}\texttt{.(}\vec{x} : \vec{\tau}\texttt{))}\ g\texttt{))} \text{ wt}}$$

$$\frac{\begin{array}{c}(R_1 : \forall\vec{\alpha}_1.\vec{\tau}_1 \rightarrow), \ldots, (R_N : \forall\vec{\alpha}_N.\vec{\tau}_N \rightarrow) \vdash \texttt{(defrel (}R_1\ \forall\vec{\alpha}_1\texttt{.(}\vec{x}_1 : \vec{\tau}_1\texttt{))}\ g_1\texttt{)} \text{ wt} \\ \vdots \\ (R_1 : \forall\vec{\alpha}_1.\vec{\tau}_1 \rightarrow), \ldots, (R_N : \forall\vec{\alpha}_N.\vec{\tau}_N \rightarrow) \vdash \texttt{(defrel (}R_N\ \forall\vec{\alpha}_N\texttt{.(}\vec{x}_N : \vec{\tau}_N\texttt{))}\ g_1\texttt{)} \text{ wt}\end{array}}{\begin{array}{c}\texttt{(defrel (}R_1\ \forall\vec{\alpha}_1\texttt{.(}\vec{x}_1 : \vec{\tau}_1\texttt{))}\ g_1\texttt{)), \ldots,} \\ \texttt{(defrel (}R_N\ \forall\vec{\alpha}_N\texttt{.(}\vec{x}_N : \vec{\tau}_N\texttt{))}\ g_N\texttt{))} \text{ wt}\end{array}}$$

Figure 3.5: Inference rules for polymorphic relation and program typing judgements.

$$
\begin{aligned}
[\![\texttt{Unit}]\!] &= \{\texttt{sole}\} \\
[\![\texttt{(Sum}\ \tau_1\ \ \tau_2\texttt{)}]\!] &= \{\texttt{(left}\ \ v\texttt{)} \mid v \in [\![\tau_1]\!]\} \cup \{\texttt{(right}\ \ v\texttt{)} \mid v \in [\![\tau_2]\!]\} \\
[\![\texttt{(Prod}\ \tau_1\ \ \tau_2\texttt{)}]\!] &= \{\texttt{(pair}\ \ v_1\ \ v_2\texttt{)} \mid v_1 \in [\![\tau_1]\!], v_2 \in [\![\tau_2]\!]\}
\end{aligned}
$$

Figure 3.6: Denotational semantics for types, again.

$$
\begin{aligned}
[\![\texttt{sole}]\!](\delta) &= \texttt{sole} \\
[\![\texttt{(left}\ \ v\texttt{)}]\!](\delta) &= \texttt{(left}\ \ [\![v]\!](\delta)\texttt{)} \\
[\![\texttt{(right}\ \ v\texttt{)}]\!](\delta) &= \texttt{(right}\ \ [\![v]\!](\delta)\texttt{)} \\
[\![\texttt{(pair}\ \ v_1\ \ v_2\texttt{)}]\!](\delta) &= \texttt{(pair}\ \ [\![v_1]\!](\delta)\ \ [\![v_2]\!](\delta)\texttt{)} \\
[\![x]\!](\delta) &= \delta(x)
\end{aligned}
$$

Figure 3.7: Denotational semantics for polymorphic values.

relation calls). Finally, note we use the shorthand $\alpha \in \tau$ to say "the type variable α occurs somewhere within type τ."

3.3 Monomorphizing Semantics

Here we update the denotational semantics to handle type variables. We use the term "concrete types" for types which do not contain type variables. We introduce *type variable substitutions*, which we apply to a types potentially containing type variables, to turn them into a concrete types Type variable substitutions walk the tree structure of the type, and replace type variables with specific concrete types.

Temporarily ignoring substitutions, imagine we wish to operate on values of an unknown type. In order to represent such a value with the array-based semantics, we need to know the size of the type so that we can have a large enough corresponding array dimension. Once we have space for the type, we can easily define equality and disequality operations on the type by using diagonals and off-diagonals, as before. This generalizes to values which may only partially consist of an unknown type. But because the specifics of the types are unknown, we have no other operations. Thus, we can treat all unknown types of the same size equivalently. As such, for each relation R with type variables $\alpha, \ldots, \beta$, we generate concrete "monomorphic" instances of these relations $R_{\sigma_1}, \ldots, R_{\sigma_n}$, corresponding to all unique-up-to-size type variable substitutions $\sigma_1, \ldots, \sigma_n$. Note that we can easily generate types of size n as a sum type of n `Unit`s, so generating these substitutions is straightforward.

$$
\begin{aligned}
[\![\texttt{(conj}\ \ g_1\ \ g_2\texttt{)}]\!](\gamma;\delta;\sigma) &= [\![g_1]\!](\gamma;\delta;\sigma) \times [\![g_2]\!](\gamma;\delta;\sigma) \\
[\![\texttt{(disj}\ \ g_1\ \ g_2\texttt{)}]\!](\gamma;\delta;\sigma) &= [\![g_1]\!](\gamma;\delta;\sigma) + [\![g_2]\!](\gamma;\delta;\sigma) \\
[\![\texttt{(fresh ((}x:\tau\texttt{))}\ \ g\texttt{)}]\!](\gamma;\delta;\sigma) &= \sum_{v \in [\![\sigma(\tau)]\!]} [\![g]\!](\gamma;\delta,x \mapsto v;\sigma) \\
[\![\texttt{(==}\ \ v_1\ \ v_2\texttt{)}]\!](\gamma;\delta;\sigma) &= \begin{cases} 1 & \text{if } [\![v_1]\!](\delta) = [\![v_2]\!](\delta) \\ 0 & \text{if } [\![v_1]\!](\delta) \neq [\![v_2]\!](\delta) \end{cases} \\
[\![\texttt{(=/=}\ \ v_1\ \ v_2\texttt{)}]\!](\gamma;\delta;\sigma) &= \begin{cases} 1 & \text{if } [\![v_1]\!](\delta) \neq [\![v_2]\!](\delta) \\ 0 & \text{if } [\![v_1]\!](\delta) = [\![v_2]\!](\delta) \end{cases} \\
[\![\texttt{(}R\ \ \vec{v}\texttt{)}]\!](\gamma;\delta;\sigma) &= \gamma(R_{\sigma'})([\![\vec{v}]\!](\delta)) \text{ where } \vec{v} : \sigma'(\vec{\tau}) \\
[\![\texttt{(factor}\ \ r\texttt{)}]\!](\gamma;\delta;\sigma) &= r
\end{aligned}
$$

Figure 3.8: Denotational semantics for polymorphic goals.

$$
\begin{gathered}
[\![\texttt{(defrel (}R_\sigma\ \ \forall\vec{\alpha}\ \ \texttt{.}\ \ \texttt{(}\vec{x}:\sigma(\vec{\tau})\texttt{)}\ \ \texttt{g)}]\!](\gamma) = (\vec{i}) \mapsto [\![g]\!](\gamma;\vec{x} \mapsto [\![\vec{i}]\!];\sigma) \\
[\![Rel_1,\ldots,Rel_Q]\!] = \text{fix } \gamma \text{ in } R_1 \mapsto [\![Rel_1]\!](\gamma),\ldots,R_Q \mapsto [\![Rel_Q]\!](\gamma)
\end{gathered}
$$

Figure 3.9: Denotational semantics for polymorphic relations and programs.

As before, γ is a relation environment and matches the relation type context. More specifically, γ is a map from relation names to multidimensional arrays. We use the term "concrete value" to refer to values constructed from `sole`, `(left` v`)`, `(right` v`)`, and `(pair` v_1 v_2`)`; concrete values can not be variables. Let δ be a value environment with type $\sigma(\Delta)$. As before, δ is a map from variable names to concrete values. The denotation for types (figure 3.6) is still as a set of specific values. We handle type variables by applying σ before taking the denotation of a type. The denotation for values (figure 3.7) under a variable context is a specific concrete value. The denotation for goals (figure 3.8) is expressed as a weight for each individual assignment of values to variables, but should be considered as a multidimensional array across all possible assignments. We assume that all needed monomorphic instances of each relation are included in the full program before fixpointing. The semantics for relations and programs are given in figure 3.9.

3.4 Non-Monomorphizing Semantics

3.4.1 Equality Patterns

In practice, semiringKanren avoids the need for monomorphization by using *equality patterns* and *large-enough instances* of polymorphic relations. Let us revisit the `sum-swap` relation:

```
(defrel (sum-swap (x : (Sum α β)) (y : (Sum β α)))
  (disj
    (fresh ((a : α))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh ((b : β))
      (conj
        (== x (right b))
        (== y (left b))))))
```

When $|\alpha| = 3$ and $|\beta| = 3$, `sum-swap` is denoted as follows (vertical axis is x, horizontal axis is y):

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

When we keep $|\alpha| = 3$ but change $|\beta| = 4$, `sum-swap` is denoted as the slightly larger array:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Which clearly has a similar structure to the $|\alpha| = |\beta| = 3$ array: diagonal sub-matrices in the top right and bottom left corners, and zeros everywhere else.

Now, imagine trying to reconstruct the second relation array from the first, only knowing its type signature. For each entry, we consider the corresponding value, and try to look up an appropriate corresponding entry in the first array. (For brevity, we use numbers subscripted with type size to denote the possible values here). For example, the top-left entry of the second array corresponds to $x = \texttt{(left 0}_3\texttt{))}$ and $y = \texttt{(left 0}_4\texttt{)}$. Any entry in the first array with $x = \texttt{(left ...)}$ and $y = \texttt{(right ...)}$ has weight 0, thus we assume this entry in the second array should also have weight 0. Jumping forward to the fifth entry in the first row: we know this corresponds to $x = \texttt{(left 0}_3\texttt{)}$ and $y = \texttt{(right 0}_3\texttt{)}$ The parts of these variables corresponding to type variable α are equal here. Similar to before, we can look up any any entry in the first array where $x = \texttt{(left } a\texttt{)}$, $y = \texttt{(right } b\texttt{)}$, and we also impose that $a = b$. These correspond to the entries along the top right diagonal of the first array. These entries all have weight 1, so we adopt that weight here. Similarly, any entry in the top-right where $a \neq b$, such as $x = \texttt{(left 1}_3\texttt{)}$ and $y = \texttt{(right 2}_3\texttt{)}$, has a weight of 0 which we adopt.

More generally, for each entry in the second array, we consider the *equality pattern* of the values at that entry, and look up an entry with the same equality pattern in the first array. When we know the non-variable-typed part of values (such as the top-level `left` or `right` in the previous example), we want to look up an entry whose non-variable-typed part is the same. When we know the variable-typed part of values, we consider whether they are equal or disequal to other variable-typed parts at the same entry, and look up an entry which has the same equalities and disequalities. Because in semiringKanren the only primitive relations on variable-typed values are direct equality or disequality, considering soley the equalities/disequalities between the variable-typed parts of a value is sufficient.

Specifically, equality patterns are defined in terms of *shells* and *holes* of value environments. The shell of a value environment is the non-variable-typed part, the holes in a value environment are the variable-typed parts. Note that holes may come from different type variables; this is important, as we can only check equality/disequality for holes of the same type variable. We say that two value environments δ_1, δ_2 have the same equality pattern, notated $\delta_1 \leftrightharpoons_\Delta \delta_2$, when

they have the same shell, and when all equal pairs of holes of a type variable in δ_1 are also equal in δ_2 (and same for disequality).

For example, consider $\delta_1 = \sigma_1(\Delta)$ and $\delta_2 = \sigma_2(\Delta)$, with Δ as follows:

$$\Delta = x : \texttt{(Sum}\ \alpha\ \alpha\texttt{)}, y : \alpha$$

We will present potential equality patterns of the form $\delta_1 \leftrightharpoons_\Delta \delta_2$, and explain why they do or do not hold. We underline holes for illustration. First consider:

$$x \mapsto \texttt{(left}\ \underline{\texttt{sole}}\texttt{)}, y \mapsto \underline{\texttt{sole}} \leftrightharpoons_\Delta x \mapsto \texttt{(right}\ \underline{\texttt{sole}}\texttt{)}, y \mapsto \underline{\texttt{sole}}$$

Here $\alpha = \texttt{Unit}$ for both δ_1 and δ_2. This equality pattern does *not* hold, because the shells of x, `(left ...)` and `(right ...)`, are not equal.

$$x \mapsto \texttt{(left}\ \underline{\texttt{1}_2}\texttt{)}, y \mapsto \underline{\texttt{0}_2} \leftrightharpoons_\Delta x \mapsto \texttt{(left}\ \underline{\texttt{1}_2}\texttt{)}, y \mapsto \underline{\texttt{1}_2}$$

Here $|\alpha| = 2$. This equality pattern does *not* hold, because the holes are disequal under δ_1 ($\texttt{1}_2 \neq \texttt{0}_2$), but equal under δ_2 ($\texttt{1}_2 = \texttt{1}_2$).

$$x \mapsto \texttt{(left}\ \underline{\texttt{1}_2}\texttt{)}, y \mapsto \underline{\texttt{0}_2} \leftrightharpoons_\Delta x \mapsto \texttt{(left}\ \underline{\texttt{0}_2}\texttt{)}, y \mapsto \underline{\texttt{1}_2}$$

Here again $|\alpha| = 2$. This equality pattern *does* hold. The shells are equal, and the holes are disequal both under δ_1 ($\texttt{1}_2 \neq \texttt{0}_2$) and under δ_2 ($\texttt{0}_2 \neq \texttt{1}_2$).

$$x \mapsto \texttt{(left}\ \underline{\texttt{0}_2}\texttt{)}, y \mapsto \underline{\texttt{0}_2} \leftrightharpoons_\Delta x \mapsto \texttt{(left}\ \underline{\texttt{sole}}\texttt{)}, y \mapsto \underline{\texttt{sole}}$$

Here $|\sigma_1(\alpha)| = 2$ and $\sigma_2(\alpha) = \texttt{Unit}$. This equality pattern *does* hold. The shells are equal, and the holes are equal both under δ_1 ($\texttt{0}_2 = \texttt{0}_2$) and under δ_2 ($\texttt{sole} = \texttt{sole}$).

3.4.2 Large-Enough Relation Instances

For `sum-swap`, we have determined that we can reproduce a call with $|\alpha| = 3, |\beta| = 4$ using an array corresponding to type variable sizes $|\alpha| = 3, |\beta| = 3$. Now we aim to do similar for `two-valued`:

```
(two-valued (x : α)
  (fresh ((y : α))
    (=/= x y)))
```

When $|\alpha| = 2$, `two-valued` is denoted as follows:

$$\begin{bmatrix} 1 & 1 \end{bmatrix}$$

If we aim to use this to reproduce the array for `two-valued` when $|\alpha| = 1$, no matter which entry we look in, we find the weight 1. Thus, the denotation we calculate for $|\alpha| = 1$ is:

$$\begin{bmatrix} 1 \end{bmatrix}$$

which indicates success. But we have already established `(two-valued sole)` (which has $|\alpha| = 1$) should fail, meaning the correct denotation should be:

$$[0]$$

To summarize, the expected behavior of `two-valued` is to fail when called with a value with type size 1, and succeed for any value with type size greater than 1. This means that we cannot use blindly the equality pattern technique everywhere. In particular, both the call site and the particular called relation instance must be *large-enough* for equality patterns to work correctly. Recall that the equality pattern process "looks up" locations in the source array where holes have the same equalities/disequalities. In the most extreme case, all holes might be disequal. Thus the size of each type variable needs to be "large-enough" to hold as many distinct values as there are possible holes. Holes correspond to type variables, so we can calculate this value by finding the maximum number of times each type variable appears in an environment, anywhere within the goal structure of the relation.

For example, in `sum-swap`, α and β both appear in the environment at most 3 times: twice from the relation arguments, and once under the `fresh`es. Thus `sum-swap` is large-enough for $|\alpha| = 3$ and $|\beta| = 3$. In `two-valued`, α appears in the environment at most twice: once from the argument, and once from the fresh. Thus `two-valued` is large-enough when $|\alpha| = 2$. When a relation call is not large-enough, we do still need to generate a monomorphized version of the called relation.

While we describe the equality pattern approach in terms of "looking up entries in the arrays," in practice we implement it by compiling polymorphic semiringKanren programs into non-polymorphic ones. In particular, polymorphic relation calls are handled in two steps. First, we call the smallest large-enough relation with fresh variables. Then, we enforce an equality pattern between the fresh variables and the original arguments. This is functionally similar to "looking up" entries: calling the relation with fresh variables brings the entries into scope, and enforcing the equality pattern makes sure we get the right weight.

3.4.3 Compiling Polymorphic Programs

Getting this to work involves some tricks. In particular, we require semiring addition (used in `disj` and `fresh`) to be idempotent. Intuitively, in the monomorphizing semantics, `fresh` may sum over different numbers of values, which could make goal weights depend on the size of type variables. For example, consider the following goal:

```
(fresh ((x : α))
  (factor 1))
```

The weight from this goal is a sum $|\alpha|$ 1s, and thus may depend on the size of α. This is something we're trying to avoid. When addition is idempotent, we

instead have:

$$[\![\texttt{(fresh ((}x : \alpha\texttt{)) (factor 1))}]\!](\dots) = 1 + \cdots + 1 = 1$$

regardless of the size of α, because idempotence gives us $1 + 1 = 1$.

3.5 Non-Monomorphizing Semantics Correctness

We now aim to prove that this compilation scheme works as intended. Throughout this section, we assume that we are working over a commutative semiring with multiplicative identity 1, additive identity and multiplicative annihlator 0, and idempotent addition (so $a + a = a$).

3.5.1 Equality Patterns Preserve Weight

Definition 2. We define a *shell* using the following grammar:

$$\begin{array}{lcl} \text{Shells} \quad s & ::= & \texttt{sole} \\ & | & \texttt{(left } s\texttt{)} \\ & | & \texttt{(right } s\texttt{)} \\ & | & \texttt{(pair } s_1 \ s_2\texttt{)} \\ & | & \texttt{(hole } \alpha\texttt{)} \end{array}$$

Shells are very similar to values, but additionally have holes which are associated with a single, non-composite type variable.

Given a concrete value with a corresponding type (including type variables), we can find its shell as follows:

$$\begin{aligned} \text{shell}_{\texttt{Unit}}(\texttt{sole}) &= \texttt{sole} \\ \text{shell}_{\texttt{(Sum } \tau_1 \ \tau_2\texttt{)}}(\texttt{(left } v\texttt{)}) &= \texttt{(left } \text{shell}_{\tau_1}(v)\texttt{)} \\ \text{shell}_{\texttt{(Sum } \tau_1 \ \tau_2\texttt{)}}(\texttt{(right } v\texttt{)}) &= \texttt{(right } \text{shell}_{\tau_2}(v)\texttt{)} \\ \text{shell}_{\texttt{(Prod } \tau_1 \ \tau_2\texttt{)}}(\texttt{(pair } v_1 \ v_2\texttt{)}) &= \texttt{(pair } \text{shell}_{\tau_1}(v_1) \ \text{shell}_{\tau_2}(v_2)\texttt{)} \\ \text{shell}_{\alpha}(v) &= \texttt{(hole } \alpha\texttt{)} \end{aligned}$$

We use $\text{holes}_{\alpha \in \tau}(v)$ to denote the values of the holes corresponding to type variable α within type τ.

$$\begin{aligned} \text{holes}_{\alpha \in \tau}(\texttt{sole}) &= \emptyset \\ \text{holes}_{\alpha \in \texttt{(Sum } \tau_1 \ \tau_2\texttt{)}}(\texttt{(left } v\texttt{)}) &= \text{holes}_{\alpha \in \tau_1}(v) \\ \text{holes}_{\alpha \in \texttt{(Sum } \tau_1 \ \tau_2\texttt{)}}(\texttt{(right } v\texttt{)}) &= \text{holes}_{\alpha \in \tau_2}(v) \\ \text{holes}_{\alpha \in \texttt{(Prod } \tau_1 \ \tau_2\texttt{)}}(\texttt{(pair } v_1 \ v_2\texttt{)}) &= \text{holes}_{\alpha \in \tau_1}(v_1), \text{holes}_{\alpha \in \tau_2}(v_2) \\ \text{holes}_{\alpha \in \alpha}(v) &= v \\ \text{holes}_{\beta \in \alpha}(v) &= \emptyset \end{aligned}$$

We also have versions of these functions which apply to *value environments* (notated as δ in the semantics) with corresponding type environments.

$$\text{envshell}_{\emptyset}() = \emptyset$$
$$\text{envshell}_{\Delta,\tau}(\vec{x} \mapsto \vec{v}, x \mapsto v) = \text{envshell}_{\Delta}(\vec{x} \mapsto \vec{v}), x \mapsto \text{shell}_{\tau}(v)$$

$$\text{envholes}_{\alpha\in\emptyset}() = \emptyset$$
$$\text{envholes}_{\Delta,\alpha\in\tau}(\vec{x} \mapsto \vec{v}, x \mapsto v) = \text{envholes}_{\alpha\in\Delta}(\vec{v}), \text{holes}_{\alpha\in\tau}(v)$$

Note that extracted holes are not labeled; we can find the corresponding `(hole ...)` in the shell by an in-order search through the shell.

Lemma 1. *Let* $\Delta \vdash v_1 : \tau$ *and* $\Delta \vdash v_2 : \tau$. *Then* $v_1 = v_2$ *iff* $\text{shell}_{\tau}(v_1) = \text{shell}_{\tau}(v_2)$ *and* $\text{holes}_{\alpha\in\tau}(v_1) = \text{holes}_{\alpha\in\tau}(v_2)\ \forall\alpha \in \tau$.

Proof. Assuming $v_1 = v_2$, by substitution we trivially have:

$$\text{shell}_{\tau}(v_1) = \text{shell}_{\tau}(v_2)$$

$$\text{holes}_{\alpha}(v_1) = \text{holes}_{\alpha}(v_2)\ \forall\alpha \in \tau$$

Thus the $\implies$ direction holds.

Now assume $\text{shell}_{\tau}(v_1) = \text{shell}_{\tau}(v_2)$ and $\text{holes}_{\alpha\in\tau}(v_1) = \text{holes}_{\alpha\in\tau}(v_2)\ \forall\alpha \in \tau$, and aim to show $v_1 = v_2$. Proof by induction on $\Delta \vdash v_1 : \tau$ and $\Delta \vdash v_2 : \tau$.

- Case: $\tau = \texttt{Unit}$. We have $\text{shell}_{\texttt{Unit}}(v_1) = \text{shell}_{\texttt{Unit}}(v_2)$. This is only possible when $v_1 = v_2 = \texttt{sole}$.
- Case: $\tau = \texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}$. We have $\text{shell}_{\texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}}(v_1) = \text{shell}_{\texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}}(v_2)$. This is only possible in two cases.
  - Subcase: $v_1 = \texttt{(left}\ v_1'\texttt{)}$ and $v_2 = \texttt{(left}\ v_2'\texttt{)}$ where $\text{shell}_{\tau_1}(v_1') = \text{shell}_{\tau_1}(v_2')$. We know:

    $$\text{holes}_{\alpha\in\texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}}(v_1) = \text{holes}_{\alpha\in\texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}}(v_2)\ \forall\alpha \in \tau$$

    so we can deduce by definition:

    $$\text{holes}_{\alpha\in\tau_1}(v_1') = \text{holes}_{\alpha\in\tau_1}(v_2')\ \forall\alpha \in \tau$$

    By the inductive hypothesis, we now know $v_1' = v_2'$. Thus $\texttt{(left}\ v_1'\texttt{)} = \texttt{(left}\ v_2'\texttt{)}$, and so $v_1 = v_2$.
  - Subcase: $v_1 = \texttt{(right}\ v_1'\texttt{)}$ and $v_2 = \texttt{(right}\ v_2'\texttt{)}$ where $\text{shell}_{\tau_2}(v_1') = \text{shell}_{\tau_2}(v_2')$. We know:

    $$\text{holes}_{\alpha\in\texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}}(v_1) = \text{holes}_{\alpha\in\texttt{(Sum}\ \tau_1\ \tau_2\texttt{)}}(v_2)\ \forall\alpha \in \tau$$

    so we can deduce by definition:

    $$\text{holes}_{\alpha\in\tau_2}(v_1') = \text{holes}_{\alpha\in\tau_2}(v_2')\ \forall\alpha \in \tau$$

    By the inductive hypothesis, we now know $v_1' = v_2'$. Thus $\texttt{(right}\ v_1'\texttt{)} = \texttt{(right}\ v_2'\texttt{)}$, and so $v_1 = v_2$.

In either case $v_1 = v_2$.

- Case: $\tau = \texttt{(Prod } \tau_1 \ \tau_2\texttt{)}$. We have:

$$\text{shell}_{\texttt{(Prod } \tau_1 \ \tau_2\texttt{)}}(v_1) = \text{shell}_{\texttt{(Prod } \tau_1 \ \tau_2\texttt{)}}(v_2)$$

This is only possible when $v_1 = \texttt{(pair } v_1' \ v_1''\texttt{)}$ and $v_2 = \texttt{(pair } v_2' \ v_2''\texttt{)}$, where $\text{shell}_{\tau_1}(v_1') = \text{shell}_{\tau_1}(v_2')$ and $\text{shell}_{\tau_2}(v_1'') = \text{shell}_{\tau_2}(v_2'')$. We also have:

$$\text{holes}_{\alpha \in \texttt{(Prod } \tau_1 \ \tau_2\texttt{)}}(v_1) = \text{holes}_{\alpha \in \texttt{(Prod } \tau_1 \ \tau_2\texttt{)}}(v_2) \ \forall \alpha \in \texttt{(Prod } \tau_1 \ \tau_2\texttt{)}$$

By definition of holes, we therefore have:

$$\text{holes}_{\alpha \in \tau_1}(v_1'), \text{holes}_{\alpha \in \tau_1}(v_1'') = \text{holes}_{\alpha \in \tau_1}(v_2'), \text{holes}_{\alpha \in \tau_1}(v_2'')$$

Because $\text{shell}_{\tau_1}(v_1') = \text{shell}_{\tau_1}(v_2')$ and $\text{shell}_{\tau_2}(v_1'') = \text{shell}_{\tau_2}(v_2'')$, we can match up the locations of holes to deduce:

$$\text{holes}_{\alpha \in \tau_1}(v_1') = \text{holes}_{\alpha \in \tau_1}(v_2') \ \forall \alpha \in \tau_1$$

$$\text{holes}_{\alpha \in \tau_2}(v_1'') = \text{holes}_{\alpha \in \tau_2}(v_2'') \ \forall \alpha \in \tau_2$$

By the inductive hypothesis, we have $v_1' = v_2'$ and $v_1'' = v_2''$. Thus, $\texttt{(pair } v_1' \ v_1''\texttt{)} = \texttt{(pair } v_2' \ v_2''\texttt{)}$, and so $v_1 = v_2$.

- Case: $\tau = \alpha$ (for any type variable). By definition we know $\text{holes}_{\alpha \in \alpha}(v_1) = v_1$ and $\text{holes}_{\alpha \in \alpha}(v_2) = v_2$. Our hypothesis holds for all type variables, so $\text{holes}_{\alpha \in \alpha}(v_1) = \text{holes}_{\alpha \in \alpha}(v_2)$. Thus by transitivity of equality, $v_1 = v_2$.

In any case, we have $v_1 = v_2$. □

Definition 3. We define $\delta_1 \leftrightharpoons_\Delta \delta_2$ when value environments δ_1 and δ_2 have the same *equality pattern* under type environment Δ.

- $\delta_1 : \sigma_1(\Delta)$ and $\delta_2 : \sigma_2(\Delta)$, for some type variable substitutions σ_1, σ_2.
- $\text{envshell}_\Delta(\delta_1) = \text{envshell}_\Delta(\delta_2)$, and
- for all type variables $\alpha \in \Delta$, for all indices i, j, holes h_1, k_1 at those indices in $\text{envholes}_{\alpha \in \Delta}(\delta_1)$, and holes h_2, k_2 at those indices in $\text{envholes}_{\alpha \in \Delta}(\delta_2)$, then $h_1 = k_1$ and $h_2 = k_2$, or $h_1 \neq k_1$ and $h_2 \neq k_2$.

Note that because the envshells for each value environment are equal, both environments must have the same number of holes in the same locations; the third condition only makes sense if the second condition holds. The third condition boils down to "if pairs of holes are equal for one value environment, then they must also be equal for the other value environment" (and similar for disequality).

Lemma 2. $\leftrightharpoons_\Delta$ *is an equivalence relation.*

Proof. $\leftrightharpoons_\Delta$ is defined as separate conditions on shells and holes. The shell condition is direct equality, so it is clearly an equivalence relation. It remains to show that the $\leftrightharpoons_\Delta$ conditions for holes forms an equivalence relation for value environments.

$\leftrightharpoons_\Delta$ is obviously reflexive; for holes h, k at type α, we either have $h = k$ and $h = k$, or $h \neq k$ and $h \neq k$. This boils down to saying that either $h = k$ or $h \neq k$; clearly one of these must be true.

Let $\delta_1 \leftrightharpoons_\Delta \delta_2$ be value environments with respective holes at indices i, j: h_1, k_1 and h_2, h_2, for type α. We have $h_1 = k_1$ and $h_2 = k_2$, or $h_1 \neq k_1$ and $h_2 \neq k_2$. "And" is symmetric, so we have $h_2 = k_2$ and $h_1 = k_1$, or $h_2 \neq k_2$ and $h_1 \neq k_1$. This applies to all pairs of holes at any indices, at for any type variable, thus we have $\delta_2 \leftrightharpoons_\Delta \delta_1$, and so $\leftrightharpoons_\Delta$ is symmetric.

Now assume $\delta_1 \leftrightharpoons_\Delta \delta_2$ and $\delta_2 \leftrightharpoons_\Delta \delta_3$, and aim to show $\delta_1 \leftrightharpoons_\Delta \delta_3$. Choose type variable α, and α-holes at indices i, j for each value environment: h_1, k_1, h_2, k_2, and h_3, k_3. There are two cases to consider: $h_1 = k_1$ and $h_2 = k_2$ and $h_3 = k_3$, or $h_1 \neq k_1$ and $h_2 \neq k_2$ and $h_3 \neq k_3$. Any other cases do not satisfy the equality pattern conditions. In the first case, we have $h_1 = k_1$ and $h_3 = k_3$. In the second case, we have $h_1 \neq k_1$ and $h_3 \neq k_3$. This applies for any arbitrarily chosen α-holes of δ_1, δ_3 at corresponding indices, for any α, thus we have $d_1 \leftrightharpoons_\Delta d_3$, and so $\leftrightharpoons_\Delta$ is transitive.

$\leftrightharpoons_\Delta$ is defined as separate conditions on shells and holes. Both conditions are reflexive, symmetric, and transitive. This, $\leftrightharpoons_\Delta$ is an equivalence relation. □

Lemma 3. *Let δ_1, δ_2 be value environments where $\delta_1 \leftrightharpoons_\Delta \delta_2$. Additionally, let $v_1, v_2 : \tau$ be values. Then $[\![v_1]\!](\delta_1) = [\![v_2]\!](\delta_1)$ if and only if $[\![v_1]\!](\delta_2) = [\![v_2]\!](\delta_2)$.*

Proof. First assume $[\![v_1]\!](\delta_1) = [\![v_2]\!](\delta_1)$ and aim to prove $[\![v_1]\!](\delta_2) = [\![v_2]\!](\delta_2)$. Note that the converse holds by symmetry, so we only need to prove one direction.

We know $\text{shell}_\tau([\![v_1]\!](\delta_1)) = \text{shell}_\tau([\![v_2]\!](\delta_1))$ by applying the trivial part of lemma 1. By assumption, $\delta_1 \leftrightharpoons_\Delta \delta_2$, so $\text{envshell}_\Delta(\delta_1) = \text{envshell}_\Delta(\delta_2)$. Because the envshells of δ_1 and δ_2 are equal, the shells of each variable in δ_1 and δ_2 are equal. So for any valid variable $x : \tau'$, $\text{shell}_{\tau'}([\![x]\!](\delta_1)) = \text{shell}_{\tau'}([\![x]\!](\delta_2))$. This cleanly extends to arbitrary values. Thus $\text{shell}_\tau([\![v_1]\!](\delta_1)) = \text{shell}_\tau([\![v_1]\!](\delta_2))$ and $\text{shell}_\tau([\![v_2]\!](\delta_1)) = \text{shell}_\tau([\![v_2]\!](\delta_2))$. By transitivity of equality, we therefore know $\text{shell}_\tau([\![v_1]\!](\delta_2)) = \text{shell}_\tau([\![v_2]\!](\delta_2))$.

For any $\alpha \in \tau$ and valid index i, we assert:

$$\text{holes}_{\alpha\in\tau}([\![v_1]\!](\delta_2))(i) = \text{holes}_{\alpha\in\tau}([\![v_2]\!](\delta_2))(i)$$

Because holes correspond to variable-typed values, and variable-typed values can only be constructed by by referencing variables in the environment, we know the values of all holes under δ_2 must come from somewhere within δ_2. Thus,

$$\text{holes}_{\alpha\in\tau}([\![v_1]\!](\delta_2))(i) = \text{envholes}_{\alpha\in\Delta}(\delta_2)(j_1)$$

$$\text{holes}_{\alpha\in\tau}([\![v_2]\!](\delta_2))(i) = \text{envholes}_{\alpha\in\Delta}(\delta_2)(j_2)$$

for some j_1, j_2. We can determine suitable j_1, j_2 by figuring out which hole of which variable the left-hand side comes from, and looking up the corresponding hole of the same variable on the right-hand side. Because we have already shown that the shells are consistent across v_1, v_2 and δ_1, δ_2, we can look up the values of the same holes under δ_1:

$$\text{holes}_{\alpha \in \tau}([\![v_1]\!](\delta_1))(i) = \text{envholes}_{\alpha \in \Delta}(\delta_1)(j_1)$$

$$\text{holes}_{\alpha \in \tau}([\![v_2]\!](\delta_1))(i) = \text{envholes}_{\alpha \in \Delta}(\delta_1)(j_2)$$

Because $[\![v_1]\!](\delta_1) = [\![v_2]\!](\delta_2)$ by assumption, we therefore have:

$$\text{holes}_{\alpha \in \tau}([\![v_1]\!](\delta_1))(i) = \text{holes}_{\alpha \in \tau}([\![v_2]\!](\delta_1))(i)$$

and so:

$$\text{envholes}_{\alpha \in \Delta}(\delta_1)(j_1) = \text{envholes}_{\alpha \in \Delta}(\delta_1)(j_2)$$

by transitivity of equality. Because of this and $\delta_1 \leftrightharpoons_\Delta \delta_2$, we therefore must also have:

$$\text{envholes}_{\alpha \in \Delta}(\delta_2)(j_1) = \text{envholes}_{\alpha \in \Delta}(\delta_2)(j_2)$$

by definition of $\leftrightharpoons_\Delta$. Therefore by transitivity of equality:

$$\text{holes}_{\alpha \in \tau}([\![v_1]\!](\delta_2))(i) = \text{holes}_{\alpha \in \tau}([\![v_2]\!](\delta_2))(i)$$

This holds for any $\alpha \in \tau$ and any valid index i, so we have:

$$\text{holes}_{\alpha \in \Delta}([\![v_1]\!](\delta_2)) = \text{holes}_{\alpha \in \Delta}([\![v_2]\!](\delta_2))$$

We have shown $\text{shell}_\tau([\![v_1]\!](\delta_2)) = \text{shell}_\tau([\![v_2]\!](\delta_2))$ and $\text{holes}_{\alpha \in \tau}([\![v_1]\!](\delta_2)) = \text{holes}_{\alpha \in \tau}([\![v_2]\!](\delta_2)\ \forall \alpha \in \tau$. Thus by lemma 1, $[\![v_1]\!](\delta_2) = [\![v_2]\!](\delta_2)$. □

Definition 4. We define the *(maximum) number of occurrences of type variable* α in type τ, denoted $\#_\alpha \tau$, as follows:

- $\#_\alpha \texttt{Unit} = 0$
- $\#_\alpha \texttt{(Sum}\ \tau_1\ \tau_2\texttt{)} = \max(\#_\alpha \tau_1, \#_\alpha \tau_2)$
- $\#_\alpha \texttt{(Prod}\ \tau_1\ \tau_2\texttt{)} = \#_\alpha \tau_1 + \#_\alpha \tau_2$
- $\#_\alpha \alpha = 1$

More specifically, this is the maximum number of α-typed holes that can occur in a value of type τ. This generalizes cleanly to type environments:

Definition 5. We define the *(maximum) number of occurrences of type variable* α in type environment Δ, denoted $\#_\alpha \Delta$, as follows:

- $\#_\alpha \emptyset = 0$
- $\#_\alpha \tau, \Delta = \#_\alpha \tau + \#_\alpha \Delta$

Definition 6. We define the *(maximum) number of occurrences of type variable α in goal g under type environment* Δ, denoted $\#_\alpha g(\Delta)$, inductively:

- $\#_\alpha$`(conj` g_1 g_2`)`$(\Delta) = \max(\#_\alpha g_1, \#_\alpha g_2)$
- $\#_\alpha$`(disj` g_1 g_2`)`$(\Delta) = \max(\#_\alpha g_1, \#_\alpha g_2)$
- $\#_\alpha$`(fresh ((`$x : \tau$`))` g`)`$(\Delta) = \#_\alpha g(\Delta, x : \tau)$
- $\#_\alpha$`(==` v_1 v_2`)`$(\Delta) = \#_\alpha \Delta$
- $\#_\alpha$`(=/=` v_1 v_2`)`$(\Delta) = \#_\alpha \Delta$
- $\#_\alpha$`(`R $\vec{v}$`)`$(\Delta) = \#_\alpha \Delta$
- $\#_\alpha$`(factor` r`)`$(\Delta) = \#_\alpha \Delta$

Which makes it easy to count for relations:

Definition 7. We define the *(maximum) number of occurrences of type variable α in relation R*, denoted $\#_\alpha R$, as the (maximum) of occurrences of α in the top-level goal, with the type environment determined from the arguments.

$$\#_\alpha\texttt{(defrel (}R\ \forall\alpha\ldots\ \texttt{(}x_1 : \tau_1\texttt{)}\ \ldots\texttt{(}x_n : \tau_1\texttt{))}\ g\texttt{)} = \#_\alpha g(x_1 : \tau_1, \ldots, x_n : \tau_n)$$

Lemma 4. *Let $\Delta \vdash v_1 : \sigma_1(\tau)$. If $|\sigma_2(\alpha)| \geq \#_\alpha \Delta, \tau\ \forall\alpha \in \Delta$, and $\delta_1 \leftrightharpoons_\Delta \delta_2$, then there exists $\Delta \vdash v_2 : \sigma_2(\tau)$ such that $\delta_1, x \mapsto v_1 \leftrightharpoons_{\Delta,\tau} \delta_2, x \mapsto v_2$.*

Proof. For the equality pattern to continue to hold, we must have $\text{shell}(v_1) = \text{shell}(v_2)$. It remains to find appropriate values for the holes of v_2.

Choose $\alpha \in \tau$, and let i be the smallest index such that $\text{envholes}_{\alpha\in\Delta,\tau}(\delta_2)[i]$ does not yet have an assigned value. For δ_1, try to find index $j < i$ such that $\text{envholes}_{\alpha\in\Delta,\tau}(\delta_1)[i] = \text{envholes}_{\alpha\in\Delta,\tau}(\delta_1)[j]$. If such j exists, assign the value of $\text{envholes}_{\alpha\in\Delta,\tau}(\delta_2)[j]$ to $\text{envholes}_{\alpha\in\Delta,\tau}(\delta_2)[i]$. If such j does not exist, choose an $\sigma_2(\tau)$-typed value which does not yet occur in $\text{envholes}_{\alpha\in\Delta,\tau}(\delta_2)$, and assign it to $\text{envholes}_{\alpha\in\Delta,\tau}(\delta_2)[i]$. Note that it is always possible to find such a value: $|\sigma_2(\alpha)| \geq \#_\alpha \Delta, \tau$, so $\sigma_2(\alpha)$ has at least as many distinct values as there are α-typed holes. □

Definition 8. Given the context $\gamma : \Gamma$ and some substitution σ, we say a monomorphic instance R_{σ_1} of relation R is *large-enough* if for all $\Delta \vdash \vec{v} : \vec{\tau}$ satisfying $\Gamma; \Delta \vdash$ `(`R $\vec{v}$`)` wt, and for all monomorphic instances R_{σ_2} of R with $\sigma_2(\alpha) \geq \sigma_1(\alpha)\ \forall\alpha \in \vec{\tau}$, and for all $\delta_1 : \sigma_1(\Delta)$, $\delta_2 : \sigma_2(\Delta)$ with $\delta_1 \leftrightharpoons_\Delta \delta_2$, the following holds:

$$[\![\texttt{(}R_{\sigma_1}\ \vec{v}\texttt{)}]\!](\gamma; \delta_1; \sigma) = [\![\texttt{(}R_{\sigma_2}\ \vec{v}\texttt{)}]\!](\gamma; \delta_2; \sigma)$$

Theorem 1. *Let $\Gamma; \Delta \vdash g$ wt, $\delta_1 : \sigma_1(\Delta)$, $\delta_2 : \sigma_2(\Delta)$, and all relations in $\gamma : \Gamma$ be large-enough. If $\delta_1 \leftrightharpoons_\Delta \delta_2$, $|\sigma_1(\alpha)| \geq \#_\alpha g(\Delta)\ \forall\alpha \in \Delta$, $|\sigma_2(\alpha)| \geq \#_\alpha g(\Delta)\ \forall\alpha \in \Delta$, and $+$ is idempotent ($a + a = a$), then $[\![g]\!](\gamma; \delta_1; \sigma_1) = [\![g]\!](\gamma; \delta_2; \sigma_2)$.*

Proof. Proof by induction on the goal structure of g.

- $g = \texttt{(conj}\ g_1\ g_2\texttt{)}$. By the inductive hypothesis:

$$[\![g_1]\!](\gamma;\delta_1;\sigma_1) = [\![g_1]\!](\gamma;\delta_2;\sigma_2)$$

$$[\![g_2]\!](\gamma;\delta_2;\sigma_2) = [\![g_2]\!](\gamma;\delta_2;\sigma_2)$$

  Thus:

$$[\![g_1]\!](\gamma;\delta_1;\sigma_1) * [\![g_2]\!](\gamma;\delta_1;\sigma_1) = [\![g_1]\!](\gamma;\delta_2;\sigma_2) * [\![g_2]\!](\gamma;\delta_2;\sigma_2)$$

  , and so $[\![g]\!](\gamma;\delta_1;\sigma_1) = [\![g]\!](\gamma;\delta_2;\sigma_2)$.

- $g = \texttt{(disj}\ g_1\ g_2\texttt{)}$. By the inductive hypothesis:

$$[\![g_1]\!](\gamma;\delta_1;\sigma_1) = [\![g_1]\!](\gamma;\delta_2;\sigma_2)$$

$$[\![g_2]\!](\gamma;\delta_2;\sigma_2) = [\![g_2]\!](\gamma;\delta_2;\sigma_2)$$

  Thus:

$$[\![g_1]\!](\gamma;\delta_1;\sigma_1) + [\![g_2]\!](\gamma;\delta_1;\sigma_1) = [\![g_1]\!](\gamma;\delta_2;\sigma_2) + [\![g_2]\!](\gamma;\delta_2;\sigma_2)$$

  and so $[\![g]\!](\gamma;\delta_1;\sigma_1) = [\![g]\!](\gamma;\delta_2;\sigma_2)$.

- $g = \texttt{(fresh ((}x:\tau\texttt{))}\ g'\texttt{)}$. $|\sigma_1(\alpha)| \geq \#_\alpha g(\Delta)\ \forall\alpha \in \Delta$, so $|\sigma_1(\alpha)| \geq \#_\alpha g'(\Delta,\tau)\ \forall\alpha \in \Delta$. By lemma 4, for any $\sigma_1(\Delta) \vdash v_1 : \sigma_1(\tau)$, there exists $\sigma_2(\Delta) \vdash v_2 : \sigma_2(\tau)$ where $\delta_1, x \mapsto v_1 \leftrightharpoons_{\Delta,\tau} \delta_2, x \mapsto v_2$. Similarly, $|\sigma_2(\alpha)| \geq \#_\alpha g'(\Delta,\tau)\ \forall\alpha \in \Delta$, so for any $\sigma_2(\Delta) \vdash v_2 : \sigma_2(\tau)$, there exists $\sigma_1(\Delta) \vdash v_1 : \sigma_1(\tau)$ where $\delta_1, x \mapsto v_1 \leftrightharpoons_{\Delta,\tau} \delta_2, x \mapsto v_2$. Thus:

$$[\![\texttt{(fresh ((}x:\tau\texttt{))}\ g'\texttt{)}]\!](\gamma;\delta_1;\sigma_1) = w_1 + \cdots + w_n$$

$$[\![\texttt{(fresh ((}x:\tau\texttt{))}\ g'\texttt{)}]\!](\gamma;\delta_2;\sigma_2) = w'_1 + \cdots + w'_m$$

  where each weight $w_1,\ldots,w_n$ appears at least once in $w'_1,\ldots,w'_m$, and each weight $w'_1,\ldots,w'_m$ appears at least once in $w_1,\ldots,w_n$. Let $r_1,\ldots,r_k$ be the unique weights appearing in both sums. We require the semiring to be commutative, so we can group identical weights together. We therefore have:

$$[\![\texttt{(fresh ((}x:\tau\texttt{))}\ g'\texttt{)}]\!](\gamma;\delta_1;\sigma_1) = (r_1 + \ldots) + \cdots + (r_k + \ldots)$$

$$[\![\texttt{(fresh ((}x:\tau\texttt{))}\ g'\texttt{)}]\!](\gamma;\delta_2;\sigma_2) = (r_1 + \ldots\ldots) + \cdots + (r_k + \ldots\ldots)$$

  We require $+$ to be idempotent. Thus:

$$[\![\texttt{(fresh ((}x:\tau\texttt{))}\ g'\texttt{)}]\!](\gamma;\delta_1;\sigma_1) = r_1 + \cdots + r_k$$

$$[\![\texttt{(fresh ((}x:\tau\texttt{))}\ g'\texttt{)}]\!](\gamma;\delta_2;\sigma_2) = r_1 + \cdots + r_k$$

$$[\![\texttt{(fresh ((}x:\tau\texttt{))}\ g'\texttt{)}]\!](\gamma;\delta_1;\sigma_1) = [\![\texttt{(fresh ((}x:\tau\texttt{))}\ g'\texttt{)}]\!](\gamma;\delta_2;\sigma_2)$$

- $g = \texttt{(==}\ v_1\ \ v_2\texttt{)}$. By lemma 3, we either have:

$$[\![v_1]\!](\delta_1) = [\![v_2]\!](\delta_1)$$

$$[\![v_1]\!](\delta_2) = [\![v_2]\!](\delta_2)$$

or

$$[\![v_1]\!](\delta_1) \neq [\![v_2]\!](\delta_1)$$

$$[\![v_1]\!](\delta_2) \neq [\![v_2]\!](\delta_2)$$

In the first case:

$$[\![\texttt{(==}\ v_1\ \ v_2\texttt{)}]\!](\gamma; \delta_1; \sigma_1) = 1$$

$$[\![\texttt{(==}\ v_1\ \ v_2\texttt{)}]\!](\gamma; \delta_2; \sigma_2) = 1$$

In the second case:

$$[\![\texttt{(==}\ v_1\ \ v_2\texttt{)}]\!](\gamma; \delta_1; \sigma_1) = 0$$

$$[\![\texttt{(==}\ v_1\ \ v_2\texttt{)}]\!](\gamma; \delta_2; \sigma_2) = 0$$

In either case, $[\![g]\!](\gamma; \delta_1; \sigma_1) = [\![g]\!](\gamma; \delta_2; \sigma_2)$.

- $g = \texttt{(=/=}\ v_1\ \ v_2\texttt{)}$. By lemma 3, we either have

$$[\![v_1]\!](\delta_1) = [\![v_2]\!](\delta_1)$$

$$[\![v_1]\!](\delta_2) = [\![v_2]\!](\delta_2)$$

or

$$[\![v_1]\!](\delta_1) \neq [\![v_2]\!](\delta_1)$$

$$[\![v_1]\!](\delta_2) \neq [\![v_2]\!](\delta_2)$$

In the first case:

$$[\![\texttt{(=/=}\ v_1\ \ v_2\texttt{)}]\!](\gamma; \delta_1; \sigma_1) = 0$$

$$[\![\texttt{(==}\ v_1\ \ v_2\texttt{)}]\!](\gamma; \delta_2; \sigma_2) = 0$$

In the second case:

$$[\![\texttt{(=/=}\ v_1\ \ v_2\texttt{)}]\!](\gamma; \delta_1; \sigma_1) = 1$$

$$[\![\texttt{(==}\ v_1\ \ v_2\texttt{)}]\!](\gamma; \delta_2; \sigma_2) = 1$$

In either case, $[\![g]\!](\gamma; \delta_1; \sigma_1) = [\![g]\!](\gamma; \delta_2; \sigma_2)$.

- $g = \texttt{(}R\ \ \vec{v}\texttt{)}$.

$$[\![\texttt{(}R\ \ \vec{v}\texttt{)}]\!](\gamma; \delta_1; \sigma_1) = \gamma(R_{\sigma'})([\![\vec{v}]\!](\delta_1))$$

$$[\![\texttt{(}R\ \ \vec{v}\texttt{)}]\!](\gamma; \delta_2; \sigma_2) = \gamma(R_{\sigma''})([\![\vec{v}]\!](\delta_2))$$

By the hypothesis that all relations in γ are large-enough, and that $\delta_1 \leftrightharpoons_{\Delta} \delta_2$, these are equivalent.

- $g =$ `(factor` k`)`.

$$[\![\texttt{(factor } k\texttt{)}]\!](\gamma;\delta_1;\sigma_1) = k = [\![\texttt{(factor } k\texttt{)}]\!](\gamma;\delta_2;\sigma_2)$$

□

The large-enough condition on γ is quite restrictive. For example, given $\Delta \vdash x : \alpha$ with $\sigma_1(\alpha) = 1$ and $\sigma_2(\alpha) = 2$, then γ is not allowed to hold relations like `two-valued`. Here, `(two-valued` x`)` would fail under δ_1 but succeed under δ_2. Ideally, we should use something smarter than $\#_\alpha g(\Delta)$, so that the type variable sizes account for these sorts of relation calls. A smarter variant may have relation calls push size requirements upward to their arguments, with a fixpoint process to allow for recursive calls. Also note that within the fixpoint process, all relations in γ start out by being large-enough: they consistently fail in all entries. This theorem lets us conclude that each fixpoint iteration preserves arrays in γ being large-enough, as long as the goal definitions of the relations only include large-enough relation calls.

3.5.2 Compiling Polymorphic Relation Calls

We now aim to compile arbitrary large-enough relation calls to instead call the smallest large-enough relation. While it remains necessary to keep monomorphized relation instances for calls smaller than the smallest large-enough instance. this allows us to remove all monomorphized instances larger than the smallest large-enough instance. In practice, implementation of this compilation scheme is verbose, so we reason here with a simplified/idealized version of the process. Rather than a formal demonstration of correctness, the following proofs aim to sketch an intuitive idea of why this method works.

We define a function to generate code to enforce equality patterns between two sub-environments. Because the generated code should work with potentially any value sub-environments, we define the function in terms of the type sub-environments. Let $\Delta_1 = \sigma_1(\Delta)$, $\Delta_2 = \sigma_2(\Delta)$, and $\alpha, \cdots \in \Delta$. For convenience, assume envshell and envholes operate on type sub-environments like Δ_1, Δ_2 by extracting shells or all possible holes as values, written in terms of ancillary

variables. In practice, this is handled manually.

```
enforce_{≒_Δ}(Δ_1; Δ_2) =
  (fresh (ancillary variables for envshell and envholes)
    (conj
      deconstruct Δ_1 and Δ_2 into the ancillary variables
      (== envshell_Δ(Δ_1) envshell_Δ(Δ_2))
      for i, j ∈ 1, ..., #_α Δ
        (disj
          (conj
            (== envholes_{α∈Δ}(Δ_1)(i) envholes_{α∈Δ}(Δ_1)(j))
            (== envholes_{α∈Δ}(Δ_2)(i) envholes_{α∈Δ}(Δ_2)(j)))
          (conj
            (=/= envholes_{α∈Δ}(Δ_1)(i) envholes_{α∈Δ}(Δ_1)(j))
            (=/= envholes_{α∈Δ}(Δ_2)(i) envholes_{α∈Δ}(Δ_2)(j))))
      ...))
```

Once we have ancillary variables to work with, the structure of the generated code directly mirrors the definition of $\leftrightharpoons_\Delta$. First, we check that the envshells of the two sub-environments are equal. Then for each pair of holes, we check that either both pairs of holes are equal, or both pairs of holes are disequal.

First note that deconstructing Δ_1 and Δ_2 into ancillary variables does not succeed when the ancillary variable values in the current environment do not match up with the current values inhabiting Δ_1 and Δ_2. When this happens, the whole `conj` fails, and so no weight is added into the summation corresponding to the outer `fresh`. This means that we can ignore these failing cases in our reasoning, and only consider cases where deconstructing Δ_1 and Δ_2 succeeds. The deconstruction process does not need to use `factor`, so it either has weight 1 or 0 (this can easily be proven by induction over goals). Thus when it succeeds it has weight 1, which is the multiplicative identity, so it does not affect the weight of the top-level `conj`. Overall, we can ignore this subgoal in our reasoning going forward, other than assuming that the environment has been properly deconstructed into ancillary variables.

Similarly note that with how envholes operates on type environments, it may process holes which are not valid given the current envshell structure. For example, consider the type environment

$$\Delta = x : \texttt{(Sum } \alpha \ \alpha\texttt{)}$$

where $\sigma_1(\alpha) = \texttt{Unit}$. Then if $\text{envshell}_{\alpha\in\Delta}(\Delta_1) = \texttt{(left (hole } \alpha\texttt{))}$, the first α-hole (at index 0 in $\text{envholes}_{\alpha\in\Delta}(\Delta_1)$) is valid, whereas the second α-hole (at index 1 in $\text{envholes}_{\alpha\in\Delta}(\Delta_1)$) is not. This second α-hole only exists in the shell `(right (hole` α`))`, which is not the current shell.

In practice, this is not an issue. It simply means that during the deconstruction process, no constraints are imposed on the ancillary variables corresponding to these holes. While the "equality patterning" part of the generated code may fail for certain assignments of these ancillary variables, it is guaranteed to succeed for at least one assignment in the outer `fresh`. It is clear that for any two variables of the same type, there must have at least one value for which they are equal. Nowhere in the generated code causes a failure for these "unconstrained" ancillary variables, so at least the first subcase of the equality patterning `disj` is guaranteed to succeed for at least one assignment. The `fresh` sums over all such assignments; our semiring is idempotent and commutative, so if there are multiple such successes, the overall weight is not affected.

Lemma 5. *Let $\delta_1 : \Delta_1 = \sigma_1(\Delta)$ and $\delta_2 : \Delta_2 = \sigma_2(\Delta)$. Then for any relation environment γ and type variable substitution σ, either:*

$$[\![\text{enforce}_{\leftrightharpoons_\Delta}(\Delta_1; \Delta_2)]\!](\gamma; \delta_1, \delta_2; \sigma) = 1$$

and $\delta_1 \leftrightharpoons_\Delta \delta_2$, or

$$[\![\text{enforce}_{\leftrightharpoons_\Delta}(\Delta_1; \Delta_2)]\!](\gamma; \delta_1, \delta_2; \sigma) = 0$$

and $\delta_1 \not\leftrightharpoons_\Delta \delta_2$.

Proof. We either have $\delta_1 \leftrightharpoons_\Delta \delta_2$, or $\delta_1 \not\leftrightharpoons_\Delta \delta_2$.

First assume $\delta_1 \leftrightharpoons_\Delta \delta_2$ and aim to show:

$$[\![\text{enforce}_{\leftrightharpoons_\Delta}(\Delta_1; \Delta_2)]\!](\gamma; \delta_1, \delta_2; \sigma) = 1$$

$\delta_1 \leftrightharpoons_\Delta \delta_2$, so $\text{envshell}_\Delta(\delta_1) = \text{envshell}_\Delta(\delta_2)$, so

$$\texttt{(==}\ \text{envshell}_\Delta(\Delta_1)\ \ \text{envshell}_\Delta(\Delta_2)\texttt{)}$$

succeeds under δ_1, δ_2. Similarly, for any α, for any holes h_1, k_1 at indices i, j in $\text{envholes}_{\alpha \in \Delta}(\delta_1)$, and holes h_2, k_2 at the same indices i, j in $\text{envholes}_{\alpha \in \Delta}(\delta_2)$, then either $h_1 = k_1$ and $h_2 = k_2$, or $h_1 \neq k_1$ or $h_2 \neq k_2$. Thus exactly one branch of the `disj` succeeds for each α, i, j (and we can ignore holes which do not "exist" here), so all such `disj`s have weight 1. We have also argued that deconstructing Δ_1, Δ_2 into ancillary variables succeeds, and relatedly that we can ignore the outer `fresh`. Thus we have a conjunction of sub-goals of weight 1, which is the multiplicative identity, so the total weight is 1.

Now assume $\delta_1 \not\leftrightharpoons_\Delta \delta_2$, and aim to show:

$$[\![\text{enforce}_{\leftrightharpoons_\Delta}(\Delta_1; \Delta_2)]\!](\gamma; \delta_1, \delta_2; \sigma) = 1$$

We know $\Delta_1 = \sigma_1(\Delta)$ and $\Delta_2 = \sigma_2(\Delta)$. $\delta_1 \leftrightharpoons_\Delta \delta_2$, so either $\text{envshell}_\Delta(\delta_1) \neq \text{envshell}_\Delta(\delta_2)$, or for some α and holes h_1, k_1 at indices i, j in $\text{envholes}_{\alpha \in \Delta}(\delta_1)$, and holes h_2, k_2 at the same indices i, j in $\text{envholes}_{\alpha \in \Delta}(\delta_2)$, we have $h_1 = k_1$ and $h_2 \neq k_2$, or $h_1 \neq k_1$ and $h_2 = k_2$.

In the first case, $\text{envshell}_\Delta(\delta_1) \neq \text{envshell}_\Delta(\delta_2)$. Thus the goal

$$\texttt{(==}\ \text{envshell}_\Delta(\Delta_1)\ \ \text{envshell}_\Delta(\Delta_2)\texttt{)}$$

fails under δ_1, δ_2. This corresponds to weight 0, which is a multiplicative annihilator. Thus the outer `conj` has weight 0, and so the generated code has weight 0 under δ_1, δ_2.

In the second case, we either have $h_1 = k_1$ and $h_2 \neq k_2$, or $h_1 \neq k_1$ and $h_2 \neq k_2$, for holes h_1, k_1 at indices i, j in $\text{envholes}_{\alpha\in\Delta}(\Delta_1)$ and holes h_2, k_2 at the same indices i, j in $\text{envholes}_{\alpha\in\Delta}(\Delta_2)$. Thus, both subcases of the corresponding `disj` fail, and so the `disj` itself fails, returning weight 0. As before, this forces the weight of the outer `conj`, and so weight of the generated code as a whole, to be 0.

In either case, if $\delta_1 \not\leftrightharpoons_\Delta \delta_2$, then $[\![\text{enforce}_{\leftrightharpoons_\Delta}(\Delta_1; \Delta_2)]\!](\gamma; \delta_1, \delta_2; \sigma) = 0$. □

Note that γ and σ are never applied in the generated code, so they have no affect on the final weight.

We can now compile relation calls. Within context $\gamma : \Gamma$, suppose that we want to compile large-enough $\texttt{(}R_{\sigma_1}\ \ \vec{v}\texttt{)}$. Let R_{σ_2} be the smallest large-enough monomorphic instance of R. When $\Delta_1 = \sigma_1(\Delta)$, $\Delta_2 = \sigma_2(\Delta)$, we use the notation $\vec{v}[\Delta_1 \mapsto \Delta_2]$ for replacing variables from sub-environment Δ_1 with corresponding variables from sub-environment Δ_2. Given $\Gamma; \Delta \vdash \texttt{(}R\ \ \vec{v}\texttt{)}$ wt, we compile as follows:

$$\begin{aligned}
&\text{compile}(\texttt{(}R_{\sigma_1}\ \ \vec{v}\texttt{)}) = \\
&\quad \texttt{(fresh ((}\delta_2 : \Delta_2 = \sigma_2(\Delta)\texttt{))} \\
&\qquad \texttt{(conj} \\
&\qquad\quad \texttt{(}R_{\sigma_2}\ \ \vec{v}[\Delta_1 \mapsto \Delta_2]\texttt{)} \\
&\qquad\quad \text{enforce}_{\leftrightharpoons_\Delta}(\Delta_1; \Delta_2)\texttt{))}
\end{aligned}$$

Theorem 2. *Given any $\gamma : \Gamma$ and σ, large-enough R_{σ_1}, $\delta_1 : \Delta_1 = \sigma_1(\Delta)$, and $\vec{v}$ satisfying $\Gamma; \Delta \vdash \texttt{(}R\ \ \vec{v}\texttt{)}$ wt, it follows:*

$$[\![\texttt{(}R_{\sigma_1}\ \ \vec{v}\texttt{)}]\!](\gamma; \delta_1; \sigma) = [\![\text{compile}(\texttt{(}R_{\sigma_1}\ \ \vec{v}\texttt{)})]\!](\gamma; \delta_1; \sigma)$$

Proof. Let $w = [\![\texttt{(}R_{\sigma_1}\ \ \vec{v}\texttt{)}]\!](\gamma; \delta_1; \sigma)$ and aim to show :

$$[\![\text{compile}(\texttt{(}R_{\sigma_1}\ \ \vec{v}\texttt{)})]\!](\gamma; \delta_1; \sigma) = w$$

First consider subcases of the outer `fresh` where $\text{enforce}_{\leftrightharpoons_\Delta}(\Delta_1; \Delta_2)$ fails. We know 0 is multiplicative annihilator, so these cases all result in weight 0.

Now consider subcases cases of the outer `fresh` where $\text{enforce}_{\leftrightharpoons_\Delta}(\Delta_1; \Delta_2)$ succeeds. Within the `fresh`, our environment is $\delta_1, \delta_2 : \Delta_1, \Delta_2$. Because we assume $\text{enforce}_{\leftrightharpoons_\Delta}(\Delta_1; \Delta_2)$ succeeds, by lemma 5 we know $\delta_1 \leftrightharpoons_\Delta \delta_2$. From this, along R_{σ_1} and R_{σ_2} being large-enough by assumption/definition, we know:

$$[\![\texttt{(}R_{\sigma_1}\ \ \vec{v}\texttt{)}]\!](\gamma; \delta_1; \sigma) = [\![\texttt{(}R_{\sigma_2}\ \ \vec{v}[\Delta_1 \mapsto \Delta_2]\texttt{)}]\!](\gamma; \delta_2; \sigma) = w$$

$\vec{v}[\Delta_1 \mapsto \Delta_2]$ does not include any variables that occur in δ_1, so the R_{σ_2} relation call has the same weight under the larger environment δ_1, δ_2:

$$[\![\texttt{(}R_{\sigma_2}\ \ \vec{v}[\Delta_1 \mapsto \Delta_2]\texttt{)}]\!](\gamma; \delta_2; \sigma) = [\![\texttt{(}R_{\sigma_2}\ \ \vec{v}[\Delta_1 \mapsto \Delta_2]\texttt{)}]\!](\gamma; \delta_1, \delta_2; \sigma) = w$$

As mentioned, we assume the enforce$_{\leftrightharpoons_\Delta}$ call succeeds. Thus:

$$[\![\texttt{(conj (}R_{\sigma_2}\ \ \vec{v}[\Delta_1 \mapsto \Delta_2]\texttt{)}\ \ \text{enforce}_{\leftrightharpoons_\Delta}(\Delta_1; \Delta_2)\texttt{)}]\!](\gamma; \delta_1, \delta_2, \sigma) = w * 1 = w$$

Our outer fresh is a sum of 0s and ws. Because our semiring is associative, has idempotent $+$, and additive identity 0, we have:

$$[\![\texttt{(fresh ((}\delta_2 : \Delta_2 = \sigma_2(\Delta)\texttt{))}\ \ \ldots\texttt{)}]\!](\gamma; \delta_1; \sigma) = (w + \ldots) + (0 + \ldots) = w + 0 = w$$

□

Theorem 2 assures us that compiled large-enough relation calls work as expected. Theorem 1 assures us that we can construct large-enough relations to begin with. In practice, not all polymorphic relations need to be large-enough. When possible, we prefer compiling relation calls as outlined here, but we can always resort to monomorphization when dealing with relation calls which are not large-enough. As such, this method is not truly "non-monomorphizing." That said, anecdotally many relation calls *do* end up being large-enough; this method is non-monomorphizing "most of the time."

Chapter 4

Related Work

4.1 miniKanren

The miniKanren[5] family of languages consists of relational programming languages which are often shallowly-embedded within a host language (such as Scheme). Relations in miniKanren are defined in terms of *goals*, which usually include: value unification, fresh variable introduction, and program branching. Languages in the miniKanren family often excel at code generation based on relational interpreters, with quine generation being a particularly compelling example. As the name may imply, semiringKanren draws extensive inspiration from miniKanren. In particular, semiringKanren draws most of its concrete syntax from the the microKanren[17] variant.

One major difference between semiringKanren and miniKanren is that semiringKanren programs are typed, whereas miniKanren programs are untyped. That said, typed variants of miniKanren do exist, such as OCanren[18] and typedKanren[19].

Some variants of miniKanren support constraints. In particular, type constraints such as `symbolo` and `numbero`, and disequality constraints (usually notated with `=/=`) are relatively common. Values in semiringKanren are typed, so semiringKanren avoids the need for specific type constraints. Disequality constraints are supported by semiringKanren, and are considerably easier to implement than in other miniKanren variants.

Implementations of miniKanren are usually based on top-down searches based on filtering and combining streams of potential solutions. With semiringKanren, we use a different approach based on bottom-up fact generation, where new facts are gradually derived from existing ones.

As of writing, semiringKanren does not support recursive data types, and therefore has not yet demonstrated the ability to generate quines.

4.2 Datalog

Datalog[14] is a family of relational programming languages based on finding the *least fixed point* of a database of facts. Relations in datalog are defined in terms of Horn clauses (disjunctions of conditions which are used to determine a fact). Datalog queries are executed by repeatedly deriving new facts using these clauses until no new facts are derived, in a process referred to as *bottom-up evaluation*. This has the advantage of being able to compute queries which may not terminate in top-down relational languages. The execution model of semiringKanren is directly based on this bottom-up approach.

While many models of relational algebra are boolean (relations either directly succeed or directly fail), some datalog variants introduce the concept of *annotated relations*. These generalize relations to K-relations, where facts are annotated with elements from a commutative semiring (a set with multiplication and addition operations, where multiplication distributes over addition). These annotations can be used to represent, for example, bag relations (where a fact may occur multiple times), or probabilistic databases (where facts may occur with some probability). In particular, it is common to use these annotations to track *provenance*, or the reasoning behind the derivation of each fact. The implementation of semiringKanren is designed to support computation over any valid semiring.

Dyna[11] is a descendent of datalog that supports *aggregators*, which permit programmers to choose from a larger set of operations on weights.

Traditionally, datalog has only a string or symbol type. Other types are constructed by defining unary relations which only succeed for a subset of symbol values[12]. In particular, this permits subtyping relations. This differs from semiringKanren, which uses algebraic data types. Some datalog implementations may also support numeric types, such as integers or floating-point numbers.

4.3 Weighted Logic Languages

Datalog is not the only relational programming language that supports annotating facts with weights. In particular, there are several variants of miniKanren which support probabilistic logic. The slpKanren[6] variant is based on stochastic logic, which generalizes stochastic grammars and requires disjunctions to be annotated with probabilities. The probKanren[32] variant allows probabilistically drawing variables from both continuous and discrete distributions, and performs probabilistic inference using a Sequential Monte Carlo method. There is also a variant based on weighted model counting[8], which tracks the likelihood of facts across several possible distributions. Of these variants, semiringKanren is most similar to slpKanren (when computing over the usual semiring for real numbers).

Probabilistic logic programming has also been widely explore outside of the miniKanren family of languages[23]. Probabilistic variants of Prolog in-

clude ProbLog, Probabilistic Horn Abduction/Independent Choice Logic, and PRISM, all of which are based on annotating disjunctions with probabilities, and have the same expressive power. Bayesian Logic Programs, CLP(BN), and the Prolog Factor language are other probabilistic Prolog variants.

4.4 Pixel Arrays

Pixel arrays[25] are a method of approximating solutions to nonlinear systems of equations. Broadly, the equations are first discretized, then plotted in arrays. Then, a generalized form of matrix multiplication to combine these arrays to reach a final result. More specifically, it aligns the matrices according to unexposed variables, multiplies them element-wise, then sums over the unexposed variable. After the process is complete, the resulting array is a plot of potential solutions, up to some level of precision depending on the initial discretization.

This method is very similar to how goals in semiringKanren are evaluated. In semiringKanren, using a fresh variable to call multiple relations in conjunction is mathematically equivalent to the generalized array multiplication method used for pixel arrays. However, the pixel array method is a discrete approximation of continuous math, and in particular may produce false positives. Because semiringKanren computes algebraic data types which are already discrete, it has no such accuracy issues. Furthermore, it can express more complicated relational joins than directly aligning variables.

4.5 SAT Compilation

The semiringKanren language supports evaluation via SAT solving, which can greatly improve efficiency[29]. FormuLog[3] is a datalog variant which uses SMT solvers to evaluate logical formulas, as an extension to the standard bottom-up datalog evaluation strategy. Other relational programming engines which support SAT evaluation include Kodkod[27], Paradox, and MACE2.

4.6 Polymorphic Logic Languages

We call a typed logic programming language *polymorphic* when it can define a single relation that can operate on several types. Polymorphic languages roughly fall into two categories: languages with *parametric* polymorphism, and languages with *ad-hoc* polymorphism. Languages with *parametric* polymorphism use *type variables* to represent unknown types. In general, operations other than assignment are not permitted on values of a variable type. Languages with *ad-hoc* polymorphism usually have some means of detecting the type of values, and using this information to potentially perform different operations depending on the type.

Relational programming languages supporting parametric polymorphism include Flix[20] (which embeds datalog), and functional IncA[22] (which extends

datalog with functions). Both of these languages use *monomorphization* to implement polymorphism, which means generating a version of each function or relation for each type it gets called with. We also support parametric polymorphism in semiringKanren, but do so *without* monomorphization—we believe this to be first for bottom-up relational programming languages. It is worth noting that Flix is also supports computing over arbitrary semirings.

Datalog variants frequently exhibit ad-hoc polymorphism. We have already discussed how datalog programs that implement types as unary relations often exhibit a form of subtyping. Implementations such as PolyDatalog[2] extend this to more sophisticated object-relational type systems.

The Mercury[24] relational programming language supports parametric polymorphism by passing along specialized unification and comparison relations for each type variable when a polymorphic relation is called. This is different from monomorphization, as only these unification and comparison relations are specialized. While the canonical implementation of Mercury is based on top-down search, Mercury has also been implemented on the bottom-up Aditi[28] deductive database system. It is unclear how exactly the implementation of polymorphism was translated, but this may be a prior instance of non-monomorphizing polymorphism for a bottom-up relational programming language.

It is worth noting that many top-down relational programming languages, including miniKanren, are untyped and thus support a form of polymorphism by default.

4.7 Other Related Work

Tabling[26] is an approach used by top-down relational programming languages in which goal results are memoized. This allows top-down languages to avoid getting stuck searching ineffectual paths, and overall function more similarly to bottom-up languages. Top-down languages are often untyped/polymorphic by default, so this approach can achieve the benefits of both polymorphic and bottom-up languages.

The paper "Testing Polymorphic Properties"[4] outlines a method to test polymorphic functions in languages with parametric polymorphism by only testing a single monomorphic instance of the function. While our work is different, we also show that only a single monomorphic instance is needed when using a polymorphic relation.

Chapter 5

Conclusions and Future Work

We have presented semiringKanren, a polymorphic bottom-up weighted relational programming language. This work builds on the state-of-the-art of relational programming in two ways: semiringKanren is the first bottom-up variant of miniKanren, and semiringKanren is the first bottom-up relational programming language to support parametric polymorphism without monomorphization. In particular, we present a denotational semantics for semiringKanren based on operations on multidimensional arrays with elements drawn from a commutative semiring. We show how these semantics can be extended to support polymorphism. Finally, we introduce the notions of *equality patterns* and *large-enough instances* of polymorphic relations, and show how these can be used to compile polymorphic programs into non-polymorphic ones, with minimal monomorphization.

We feel that the miniKanren approach to relational programming lends naturally to these developments. In particular, microKanren-style `conj` and `disj`-based syntax leads to a very straightforward denotational semantics. Furthermore, the flexibility to freely nest goals makes it straightforward to express the nontrivial patterns used when compiling polymorphic programs into non-polymorphic ones.

We hope our new method of implementing polymorphism may find applications in relational languages beyond semiringKanren. We believe it may offer efficiency gains for polymorphic programs by reducing the need to recalculate the same relation at different types. We also believe it can help enable "separate evaluation" of programs, so that relations can be pre-computed and reused without reevaluation.

The OCaml implementation of semiringKanren is available on GitHub[1].

[1] semiringKanren repository: `https://github.com/sporkl/semiringkanren`

5.1 Future Work

5.1.1 Large-Enough Relation Instances

In this work, we present a method of counting type variable occurrences to determine when a relation is large-enough, but acknowledge that it is not sufficient in all cases. It remains to find a fully-general method for calculating the necessary type variable sizes for a relation to be large-enough. As we have briefly discussed, such a method may involve relation calls providing information about their own argument size requirements during the calculation process. In order to handle recursive calls, a fixpoint process may me necessary (as it is in normal program execution).

5.1.2 Higher-Order Relations

It is relatively rare for relational programming languages to support higher-order relations (relations which take other relations as arguments). Existing work in the space includes λProlog[10] and λKanren[31].

With the `option-map` example, we have shown that higher-order relations can be encoded as products if they are used affinely (they can be called either zero times or once). It is not immediately clear how powerful this technique is. In particular, once a relation is encoded as a product, it may be possible to effectively duplicate it (likely using a new language construct) to get more calls.

5.1.3 Recursive Types

Thus far, semiringKanren does not support recursive types. This is very limiting. For example, semiringKanren cannot represent arbitrary lambda calculus terms and thus perform the traditional quine generation of miniKanren languages. In practice, recursive types should be able to be arbitrarily large, which seems incompatible with the finite array semantics semiringKanren currently uses.

We are aware of three potential ways to bring recursive data types to semiringKanren. First, if semiringKanren gains full support for higher-order relations, it may become possible to express recursive types using Church-style encodings (given that semiringKanren already supports polymorphism).

Secondly, we may be able to support *gas-recursive types*, which are recursive types equipped with an additional size parameter. This may make it possible to calculate maximum sizes for types, so we can find an upper bound on the sizes of the relation arrays. This is analogous to the "vector" type common in dependently-typed programming languages, where the type holds a number corresponding to the length.

Finally, it may be possible to use a system based on defunctionalization and refunctionalization to eliminate recursive data types, as in "Exact Recursive Probabilistic Programming"[7]. From a preliminary investigation, it appears possible to translate the de/refunctionalized programs presented in that paper to

semiringKanren. However, it remains to be seen if the approach fully generalizes to "derelationalization" and "rerelationalization" for arbitrary semiringKanren programs.

5.1.4 Semiring Fixpoint Convergence

It is not yet clear when semiringKanren programs terminate, and when they do not. Relatedly, it is not clear whether or not semiringKanren is Turing-complete. Of course, these sorts of characteristics may depend on the specific semiring in use.

There is a large body of related work about datalog in the sphere. For example, it is known that the base datalog language always terminates and can express any polynomial-time algorithm[20], but this does not hold for many datalog variants. Datalog variants computing over provenance semirings are also known to terminate[14]. It has been proven that certain algebraic properties of semirings correspond to convergence properties in datlog[1]. It remains to see whether these hold for semiringKanren.

Of particular interest is the ability to negate goals. For semirings which are also rings (have additive inverses), we can express "not" with `(factor -1)`. For example, for the usual semiring over real numbers, we can express disequality as follows:

```
(defrel (=/= (x : α) (y : α))
  (disj
    (factor 1)
    (conj
      (factor -1)
      (== x y))))
```

This roughly reads as "succeed always, but negate the success when $x = y$." Manually computing, this gives us an off-diagonal matrix as expected for `=/=`. While this particular example terminates, programs with negation in general may not. It is also unclear what the "meaning" of negation is here. Relational programming languages supporting negation are uncommon, and use a variety of approaches. Negation has been implemented in Datalog using a three-valued semiring, and with "stratified" approaches (where different rules are evaluated at different times) to evaluation. Several miniKanren variants support `noto` goals, including stableKanren[16] which is based on stable model semantics, and minnaKanren[9] which uses a system of constraints.

5.1.5 SMT Solving

We have shown that semiringKanren programs can be translated into SAT problems for efficient evaluation over the boolean semiring. We believe this approach can be extended to also use SMT solvers, for evaluation over other semirings.

5.1.6 Applications

It remains to see where semiringKanren may be effectively applied. We believe semiringKanren may be a strong fit for probabilistic modelling. It may also be worthwhile to explore existing applications of datalog, and see which may benefit from the greater expressivity of miniKanren-style syntax.

5.1.7 Alternative Evaluation Strategies

We have shown evaluation strategies for semiringKanren based on operations on semiring arrays, and translate to SAT problems. We believe there are several other evaluation strategies worth exploring.

For semiringKanren's array-based evaluation, the arrays it uses are highly structured. The diagonal and off-diagonal arrays used for equality and disequality are a clear example. Thus, using dense arrays is likely an unnecessary use of space. There are methods which exploit these sorts of structures in arrays/tensors to enable parallelization and achieve significant speedups[13]. Implementing a backend based on such a system may make larger problems and programs tractable for semiringKanren.

We already have an practical method for compiling semiringKanren programs to SAT problems[29]. We can compile boolean formulas into quantum circuits, thus we can use Grover's database search algorithm[15] to perform this evaluation.

Bibliography

[1] Mahmoud Abo Khamis et al. "Convergence of datalog over (Pre-) Semirings". In: *J. ACM* 71.2 (Apr. 2024). ISSN: 0004-5411. DOI: 10.1145/3643027. URL: https://doi.org/10.1145/3643027.

[2] Paolo Atzeni, Giorgio Gianforme, and Daniele Toti. "Polymorphism in Datalog and Inheritance in a Metamodel". In: *Foundations of Information and Knowledge Systems*. Ed. by Sebastian Link and Henri Prade. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 114–132. ISBN: 978-3-642-11829-6.

[3] Aaron Bembenek and Stephen Chong. *FormuLog: Datalog for static analysis involving logical formulae*. 2018. arXiv: 1809.06274 [cs.PL]. URL: https://arxiv.org/abs/1809.06274.

[4] Jean-Philippe Bernardy, Patrik Jansson, and Koen Claessen. "Testing Polymorphic Properties". In: *Programming Languages and Systems*. Ed. by Andrew D. Gordon. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 125–144. ISBN: 978-3-642-11957-6.

[5] William Byrd, Eric Holk, and Daniel Friedman. "miniKanren, live and untagged: quine generation via relational interpreters (programming pearl)". In: (Sept. 2012). DOI: 10.1145/2661103.2661105.

[6] William E. Byrd and Rebecca Swords. *slpKanren*. 2013. URL: https://github.com/webyrd/slpKanren.

[7] David Chiang, Colin McDonald, and Chung-chieh Shan. "Exact Recursive Probabilistic Programming". In: *Proc. ACM Program. Lang.* 7.OOPSLA1 (Apr. 2023). DOI: 10.1145/3586050. URL: https://doi.org/10.1145/3586050.

[8] Evan Donahue. "Efficient Variational Inference in miniKanren with Weighted Model Counting". In: *miniKanren and Relational Programming Workshop*. 2022.

[9] Evan Donahue. "Goals as Constraints: Writing miniKanren Constraints in miniKanren". In: *Proceedings of the 2023 miniKanren and Relational Programming Workshop*. Ed. by Nada Amin and William E. Byrd. Cambridge: Harvard Technical Report, 2023, pp. 1–12.

[10] Amy Felty et al. "λProlog: An extended logic programming language". In: *9th International Conference on Automated Deduction*. Ed. by Ewing Lusk and Ross Overbeek. Berlin, Heidelberg: Springer Berlin Heidelberg, 1988, pp. 754–755. ISBN: 978-3-540-39216-3.

[11] Matthew Francis-Landau. "Declarative Programming via Term Rewriting". PhD thesis. Johns Hopkins University, 2024. URL: https://matthewfl.com/papers/mfl-dissertation.pdf.

[12] T. Fruhwirth et al. "Logic programs as types for logic programs". In: *[1991] Proceedings Sixth Annual IEEE Symposium on Logic in Computer Science*. 1991, pp. 300–309. DOI: 10.1109/LICS.1991.151654.

[13] Mahdi Ghorbani et al. "Compressed and Parallelized Structured Tensor Algebra". In: *Proc. ACM Program. Lang.* 9.OOPSLA1 (Apr. 2025). DOI: 10.1145/3720506. URL: https://doi.org/10.1145/3720506.

[14] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. "Provenance semirings". In: *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. PODS '07. Beijing, China: Association for Computing Machinery, 2007, pp. 31–40. ISBN: 9781595936851. DOI: 10.1145/1265530.1265535. URL: https://doi.org/10.1145/1265530.1265535.

[15] Lov K. Grover. "A fast quantum mechanical algorithm for database search". In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*. STOC '96. Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, pp. 212–219. ISBN: 0897917855. DOI: 10.1145/237814.237866. URL: https://doi.org/10.1145/237814.237866.

[16] Xiangyu Guo, James Smith, and Ajay Bansal. "stableKanren: Integrating Stable Model Semantics with miniKanren". In: *Proceedings of the 25th International Symposium on Principles and Practice of Declarative Programming*. PPDP '23. Lisboa, Portugal: Association for Computing Machinery, 2023. ISBN: 9798400708121. DOI: 10.1145/3610612.3610617. URL: https://doi.org/10.1145/3610612.3610617.

[17] Jason Hemann and Daniel P. Friedman. "Nearly Macro-free microKanren". In: *Trends in Functional Programming*. Ed. by Stephen Chang. Cham: Springer Nature Switzerland, 2023, pp. 72–91. ISBN: 978-3-031-38938-2.

[18] Dmitrii Kosarev and Dmitry Boulytchev. "Typed Embedding of a Relational Language in OCaml". In: (2018). DOI: 10.48550/ARXIV.1805.11006. URL: https://arxiv.org/abs/1805.11006.

[19] Nikolai Kudasov and Artem Starikov. *typedKanren: Statically Typed Relational Programming with Exhaustive Matching in Haskell*. 2024. DOI: 10.48550/ARXIV.2408.03170. URL: https://arxiv.org/abs/2408.03170.

[20] Magnus Madsen and Ondřej Lhoták. "Fixpoints for the masses: programming with first-class Datalog constraints". In: *Proc. ACM Program. Lang.* 4.OOPSLA (Nov. 2020). DOI: 10.1145/3428193. URL: https://doi.org/10.1145/3428193.

[21] Raymond J. Mooney. "Inductive logic programming for natural language processing". In: *Inductive Logic Programming*. Ed. by Stephen Muggleton. Berlin, Heidelberg: Springer Berlin Heidelberg, 1997, pp. 1–22. ISBN: 978-3-540-69583-7.

[22] André Pacak and Sebastian Erdweg. "Functional Programming with Datalog". In: *36th European Conference on Object-Oriented Programming (ECOOP 2022)*. Ed. by Karim Ali and Jan Vitek. Vol. 222. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, 7:1–7:28. ISBN: 978-3-95977-225-9. DOI: 10.4230/LIPIcs.ECOOP.2022.7. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2022.7.

[23] Fabrizio Riguzzi. *Foundations of probabilistic logic programming*. en. River Publishers Series in Software Engineering. London, England: Taylor & Francis, Sept. 2022.

[24] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. "The execution algorithm of mercury, an efficient purely declarative logic programming language". In: *The Journal of Logic Programming* 29.1 (1996). High-Performance Implementations of Logic Programming Systems, pp. 17–64. ISSN: 0743-1066. DOI: https://doi.org/10.1016/S0743-1066(96)00068-4. URL: https://www.sciencedirect.com/science/article/pii/S0743106696000684.

[25] David I. Spivak et al. *Pixel Arrays: A fast and elementary method for solving nonlinear systems*. 2017. arXiv: 1609.00061 [math.NA]. URL: https://arxiv.org/abs/1609.00061.

[26] Hisao Tamaki and Taisuke Sato. "OLD resolution with tabulation". In: *Third International Conference on Logic Programming*. Ed. by Ehud Shapiro. Berlin, Heidelberg: Springer Berlin Heidelberg, 1986, pp. 84–98. ISBN: 978-3-540-39831-8.

[27] Emina Torlak and Daniel Jackson. "Kodkod: A Relational Model Finder". In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by Orna Grumberg and Michael Huth. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 632–647. ISBN: 978-3-540-71209-1.

[28] Jayen Vaghanl et al. "The Aditi deductive database system". In: *The VLDB Journal* 3.2 (1994), pp. 245–288.

[29] Dmitri Volkov, Yafei Yang, and Chung chieh Shan. *Committing to the bit: Relational programming with semiring arrays and SAT solving*. 2025. arXiv: 2509.22614 [cs.PL]. URL: https://arxiv.org/abs/2509.22614.

[30] John Von Neumann et al. "Various techniques used in connection with random digits". In: *John von Neumann, Collected Works* 5.768-770 (1963), p. 1.

[31] MA Weixi, Kuang-Chen Lu, and Daniel P. Friedman. "Higher-order Logic Programming with λ Kanren". In: 2020. URL: https://minikanren.org/workshop/2020/minikanren-2020-paper4.pdf.

[32] Robert Zinkov and William E Byrd. "probKanren: A Simple Probabilistic Extension for microKanren." In: *ICLP Workshops*. 2021.